

CS1050 – Computer Organization and Digital Design

Lab 10-Nano Processor Design Competition

Group 15

- **240136L – M. Dinojan**
- **240107B - De Zoysa P.C.A**
- **240118J – Dhananjaya P.D.L.K**
- **240710R – W.W.N.D.Wickramarathna**

Introduction

We had to design a nano processor as our task, using the components we learned in our labs. This is a four-member group project. We designed 2 versions of our nano processor. First, we designed the nano processor with the minimum requirements mentioned in the lab sheet. They are,

- 4-bit Add/Subtract unit
- 3-bit adder
- 3-bit Program Counter
- 8-way 2-bit multiplexers
- 2-way 3-bit multiplexers
- 2-way 4-bit multiplexers
- Register Bank
- Program ROM
- Instruction Decoder

After that we updated our processor with 4-bit Add/Subtract unit with the ALU and we used 4-bit comparator, 4-bit Add/Subtract unit, 2-bit multiplier, and a logic operation unit inside the ALU. Moreover, we designed 4bit program counter, 16 locations ROM, 4bit adder also. Our nano processor version 1 can execute up to 8 instructions for different tasks and version 2 can execute up to 16. Then we used an ALU to implement summation, subtraction, and multiplication as arithmetic operations and logical operations. Such as bitwise OR, bitwise AND etc. in our extended version.

Nano processor version 01

Components and instructions

1. Allocated reset button

- The nano processor can be reset by pressing the btnC.
- The clock speed of the internal clock of the machine was reduced from 100MHz to 0.5MHz using a slow clock to make the calculation process visible to the naked eye.
- In order to reset the nano processor, it is necessary to press and hold the reset button for at least 2 seconds.

2. LED signal

- LED0-LED3 Output of R7 register in Register Bank
- LED0-LED3 → Outputs a 4-bit number as a signed number in two's complement method
- LED12-LED15 Flags of 4-bit Add/Subtract Unit
 - LED12 → Sign Flag
 - LED13 → Overflow Flag
 - LED14 → Zero Flag
 - LED15 → Carry Flag

3. 7-Segment Display

- The rightmost segment of the 7-Segment display is used to display the magnitude of the output from the R7 register in the Register Bank.

4. Expected behavior of the program

- After the calculation is performed the process will be held because of the jump instruction implemented.
- Therefore, it is necessary to manually reset the nano processor to perform the instructions hardcoded in the ROM.

Slice logic and Design Primitives

Slice Logic

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	75	0	20800	0.36
LUT as Logic	75	0	20800	0.36
LUT as Memory	0	0	9600	0.00
Slice Registers	49	0	41600	0.12
Register as Flip Flop	49	0	41600	0.12
Register as Latch	0	0	41600	0.00
F7 Muxes	0	0	16300	0.00
F8 Muxes	0	0	8150	0.00

Design primitives

7. Primitives

Ref Name	Used	Functional Category
FDRE	37	Flop & Latch
LUT2	32	LUT
LUT4	25	LUT
LUT3	22	LUT
OBUF	19	IO
FDCE	12	Flop & Latch
LUT5	10	LUT
LUT6	9	LUT
CARRY4	8	Carry Logic
LUT1	2	LUT
IBUF	2	IO
BUFG	1	Clock

Executable Instruction Set

Instruction	Description	Format (12-bit instruction)
MOVI R,d	Move(11 downto 10 ==10) immediate value d(3 downto 0) to register R (9 downto 7).	1 0 RRR 000 dddd
ADD Ra,Rb	Add (11 downto 10 ==00) values in registers Ra(9 downto 7) and Rb(6 downto 4) and store the result in Ra.	0 0 RaRaRa RbRbRb 0000
NEG R	Get negation(11 downto 10 ==01) of the value that store in the register R(9 downto 7) and restore the created negation to R register.	0 1 RRR 0 0 0 0 0 0 0
JZR R,d	Jump(11 downto 10 ==11) to Pc to value d(2 downto 0) if value in register R(9 downto 7) is 0. i.e, If R==0; Pc⇒d; Else; Pc⇒ Pc + 1;	1 1 RRR 0 0 0 0 d d d

Assembly Code for The Design

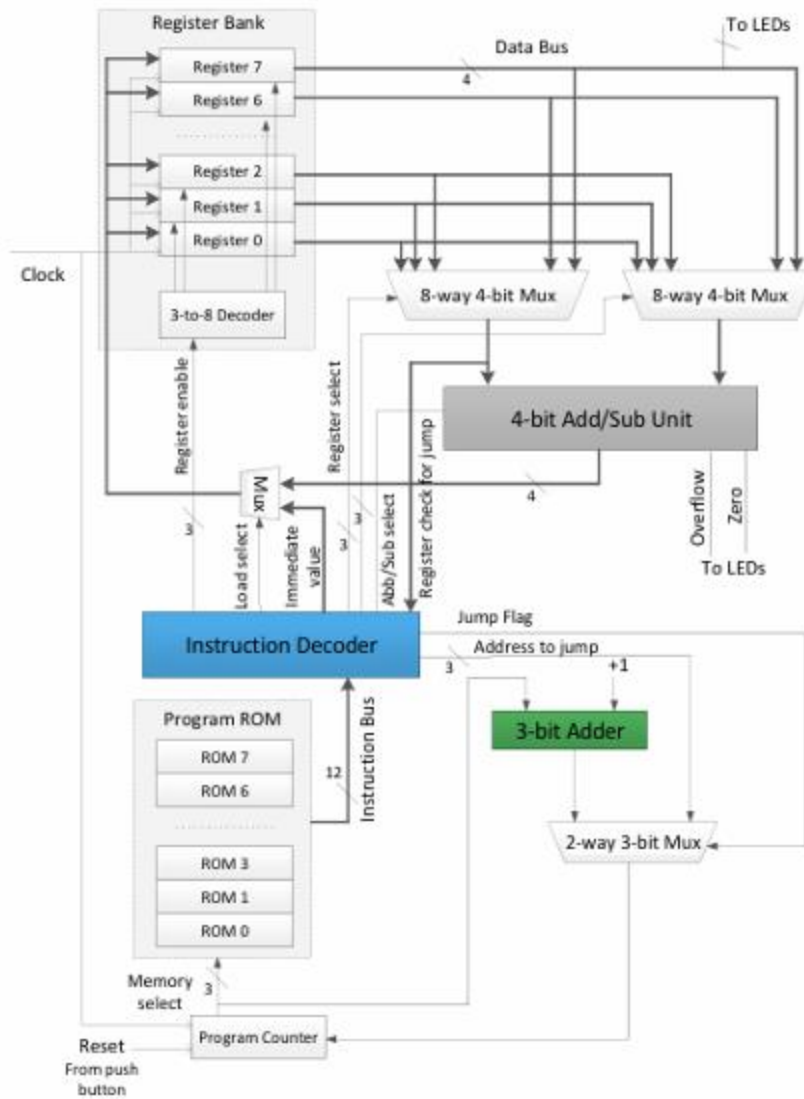
```

;      process => 3 + 2 + 1 = 6

MOVI R1,4      ; R1 <- 4
MOVI R2,1      ; R2 <- 1
NEG R2         ; R2 <- -1
ADD R1,R2      ; R1 <- R2 + R1 3
JZR R1,7       ; Jump to line 8 if R1 = 0
ADD R7,R1      ; R7 <- R7 + R1 5
JZR R0,3       ; Jump to line 4 if R0 = 0
JZR R0,7       ; Jump to line 8 if R0 = 0

```

Design Components



1. Nano Processor

Design source VHDL code of Nano Processor

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity NanoProcessor is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          reg : out STD_LOGIC_VECTOR (3 downto 0);
          zero : out STD_LOGIC;
          overflow : out STD_LOGIC;
          carry : out STD_LOGIC);
end NanoProcessor;

architecture Behavioral of NanoProcessor is

    -- Signals
    signal O, b, Add_out : STD_LOGIC_VECTOR(2 downto 0);
    signal Ins_Bus : STD_LOGIC_VECTOR(11 downto 0);

    signal Load_Select, Add_Sub_Selector, Jump_Flag : STD_LOGIC;
    signal Reg_Enable, Reg_Select_1, Reg_Select_2 : STD_LOGIC_VECTOR(2
downto 0);
    signal Address : STD_LOGIC_VECTOR(2 downto 0);

    signal R0,R1,R2,R3,R4,R5,R6,R7 : STD_LOGIC_VECTOR(3 downto 0);
    signal Y1,Y2,S,X,Imm_Value : STD_LOGIC_VECTOR(3 downto 0);

    signal Z_out, Overflow_0 : STD_LOGIC;

begin

    -- PC
    PC: entity work.PC_3_bit
    port map(D=>b, Clk=>clk, Res=>reset, Q=>O);

    -- ROM
    ROM: entity work.Program_ROM
    port map(address=>O, instruction_code=>Ins_Bus);

    -- Decoder
    DEC: entity work.Instruction_Decoder
    port map(I=>Ins_Bus,
```

```

        Reg_Check_Jump=>Y1,
        Load_Select=>Load_Select,
        Imm_Value=>Imm_Value,
        Reg_Enable=>Reg_Enable,
        Reg_Select_1=>Reg_Select_1,
        Reg_Select_2=>Reg_Select_2,
        Add_Sub=>Add_Sub_Selector,
        Jump_Flag=>Jump_Flag,
        Address=>Address);

-- Register Bank
REGS: entity work.Register_Bank
port map(Clk=>clk, Reset=>reset, D=>X, I=>Reg_Enable,
        R0=>R0,R1=>R1,R2=>R2,R3=>R3,
        R4=>R4,R5=>R5,R6=>R6,R7=>R7);

-- MUX Read
MUX1: entity work.MUX_8_way_4_bit
port map(Reg_Select_1,R0,R1,R2,R3,R4,R5,R6,R7,Y1);

MUX2: entity work.MUX_8_way_4_bit
port map(Reg_Select_2,R0,R1,R2,R3,R4,R5,R6,R7,Y2);

-- ALU
ALU: entity work.ADD_SUB_4bit
port map(Y1,Y2,Add_Sub_Selector,carry,S,Overflow_0,Z_out);

-- MUX for write
MUXW: entity work.MUX_2_way_4_bit
port map(Load_Select,S,Imm_Value,X);

-- PC +1
ADD3: entity work.Adder_3bit
port map(O,Add_out,open);

-- Jump MUX
MUXPC: entity work.MUX_2_way_3_bit
port map(Jump_Flag,Add_out,Address,b);

-- Outputs
reg <= R7;
zero <= Z_out;
overflow <= Overflow_0;

end Behavioral;

```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Nanoprocessor is
-- Port ( );
end TB_Nanoprocessor;
```

```

architecture Behavioral of TB_Nanoprocessor is

component NanoProcessor
    Port (clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          reg : out STD_LOGIC_VECTOR (3 downto 0);
          display : out STD_LOGIC_VECTOR (6 downto 0);
          zero : out STD_LOGIC;
          overflow : out STD_LOGIC;
          anode : out STD_LOGIC_VECTOR (3 downto 0);
          carry: out STD_LOGIC );
end component;

Signal clk,reset,zero,overflow,carry : STD_LOGIC;
signal reg,anode : STD_LOGIC_VECTOR (3 downto 0);
signal display : STD_LOGIC_VECTOR (6 downto 0);

begin

    UUT : NanoProcessor
        Port Map (

```



```

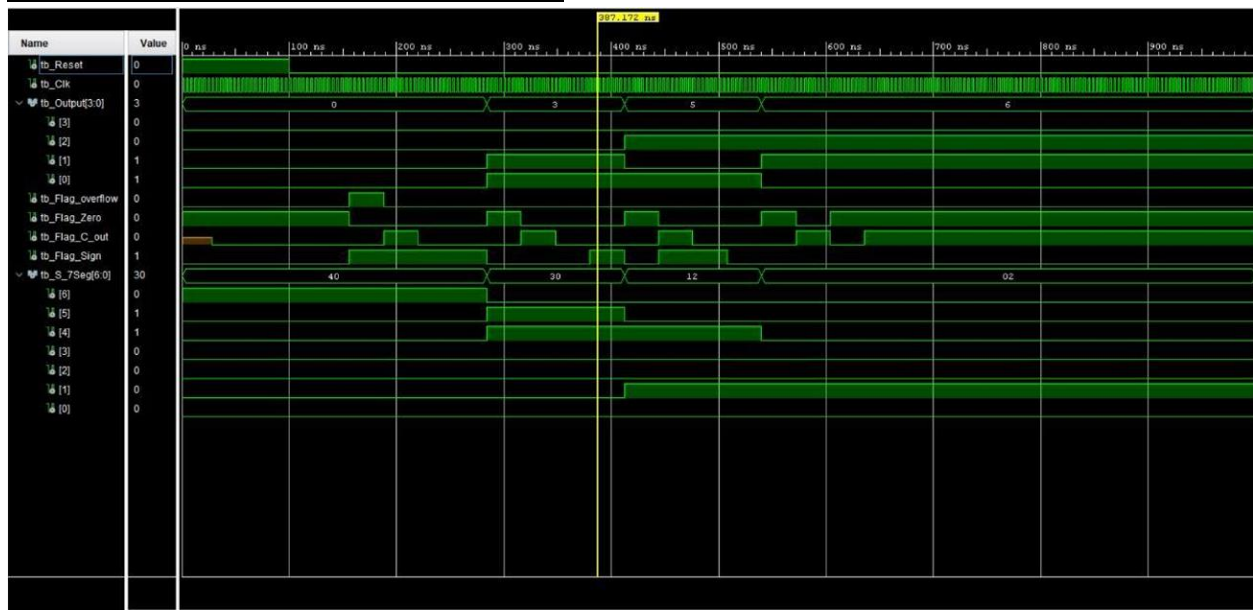
        clk => clk,
        reset => reset,
        reg => reg,
        zero => zero,
        overflow => overflow,
        carry => carry,
        display => display,
        anode => anode
    );

process
begin
    while true loop
        clk <= '0';
        wait for 50 ns;
        clk <= '1';
        wait for 50 ns;
    end loop;
end process;

process
begin
    reset <= '1';
    wait for 100 ns;
    reset <= '0';
    wait for 2000 ns;
    wait;
end process;

end Behavioral;
```

Timing Diagram for Nano processor:



2. Half Adder

Design source VHDL code of Half Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HA is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          S : out STD_LOGIC;
          C : out STD_LOGIC);
end HA;

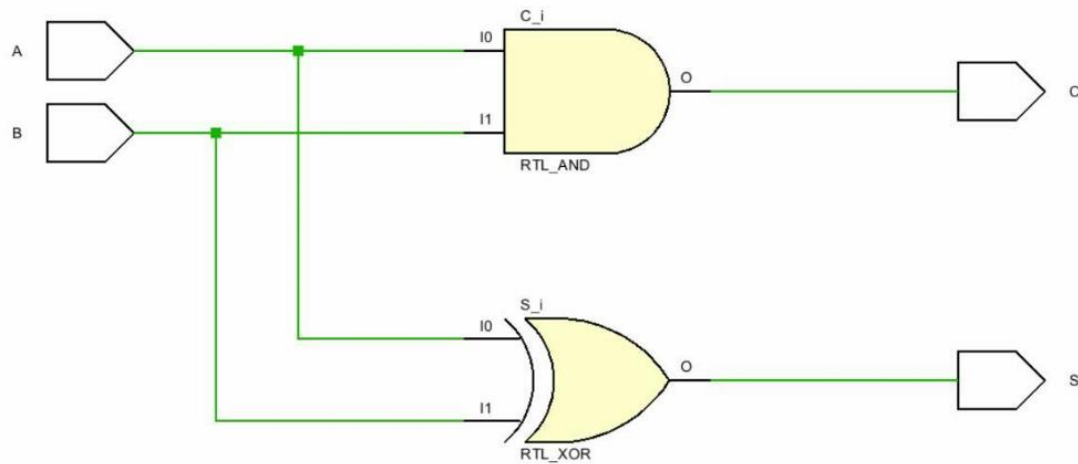
architecture Behavioral of HA is

begin

    S <= A XOR B;
    C <= A AND B;

end Behavioral;
```

RTL Schematic diagram of Half Adder



3. Full Adder

Design source VHDL code of Full Adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          C_in : in STD_LOGIC;
          S : out STD_LOGIC;
          C_out : out STD_LOGIC);
end FA;

architecture Behavioral of FA is

    COMPONENT HA PORT
        ( A, B: IN STD_LOGIC;
          S, C: OUT STD_LOGIC );
    END COMPONENT;

    SIGNAL HA0_S, HA0_C, HA1_S, HA1_C: STD_LOGIC;

begin

    HA0: HA PORT MAP
        ( A => A,
          B => B,
          S => HA0_S,
```

```

        C => HA0_C );

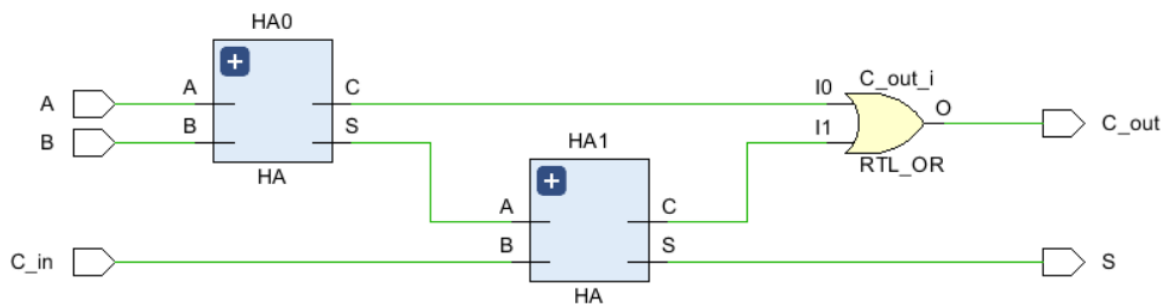
    HA1: HA PORT MAP
        ( A => HA0_S,
          B => C_in,
          S => HA1_S,
          C => HA1_C );

    S <= HA1_S;
    C_out <= HA0_C OR HA1_C;

end Behavioral;

```

RTL Schematic diagram of Full Adder



4. Ripple Carry Adder

Design source VHDL code of RCA

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4 is
    Port (
        A      : in  STD_LOGIC_VECTOR(3 downto 0);
        B      : in  STD_LOGIC_VECTOR(3 downto 0);
        C_in   : in  STD_LOGIC;
        S      : out STD_LOGIC_VECTOR(3 downto 0);
        C_out  : out STD_LOGIC
    );
end RCA_4;

```

architecture Behavioral of RCA_4 is

```
COMPONENT FA
  PORT(
    A      : in  STD_LOGIC;
    B      : in  STD_LOGIC;
    C_in   : in  STD_LOGIC;
    S      : out STD_LOGIC;
    C_out  : out STD_LOGIC
  );
END COMPONENT;
```

```
SIGNAL C1, C2, C3 : STD_LOGIC;
```

begin

```
FA0: FA PORT MAP(
  A => A(0),
  B => B(0),
  C_in => C_in,
  S => S(0),
  C_out => C1);
```

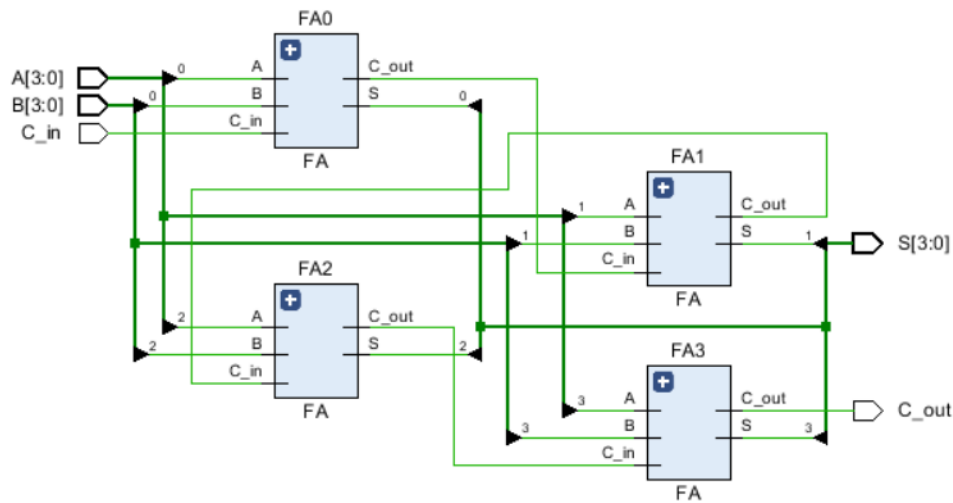
```
FA1: FA PORT MAP(
  A => A(1),
  B => B(1),
  C_in => C1,
  S => S(1),
  C_out => C2);
```

```
FA2: FA PORT MAP(
  A => A(2),
  B => B(2),
  C_in => C2,
  S => S(2),
  C_out => C3);
```

```
FA3: FA PORT MAP(
  A => A(3),
  B => B(3),
  C_in => C3,
  S => S(3),
  C_out => C_out);
```

end Behavioral;

RTL Schematic diagram of RCA



5. 2-way-3-bit Multiplexer Design source VHDL code

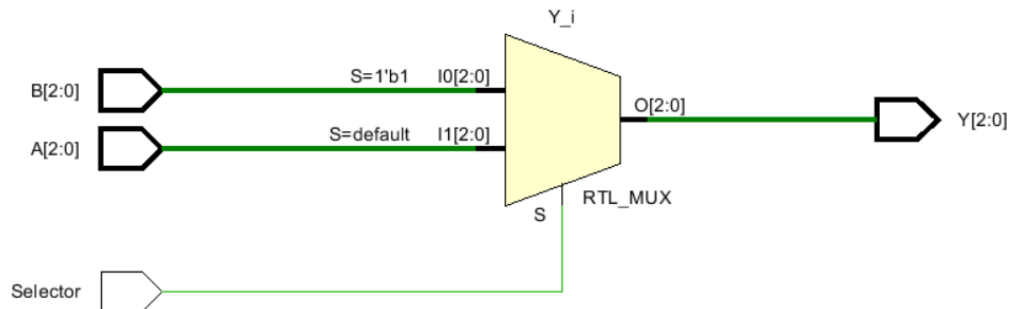
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2_way_3_bit is
    Port ( Selector : in STD_LOGIC;
          A : in STD_LOGIC_VECTOR (2 downto 0);
          B : in STD_LOGIC_VECTOR (2 downto 0);
          Y : out STD_LOGIC_VECTOR (2 downto 0));
end MUX_2_way_3_bit;

architecture Behavioral of MUX_2_way_3_bit is

begin
    Y <= B when (Selector = '1') else A;
end Behavioral;
```

RTL Schematic Diagram of 2way3 bit Multiplexer



Simulation source code for 2 way 3 bit Multiplexer

```

library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX_2_way_3_bit is -- Port ( ); end
TB_MUX_2_way_3_bit;

architecture Behavioral of TB_MUX_2_way_3_bit is component
MUX_2_way_3_bit PORT ( Selector: in STD_LOGIC; A : in
STD_LOGIC_VECTOR(2 downto 0); B : in STD_LOGIC_VECTOR(2 downto
0); Y : out STD_LOGIC_VECTOR(2 downto 0)); end component;

signal Selector : std_logic; signal A,B,Y : std_logic_vector(2
downto 0);

begin UUT : MUX_2_way_3_bit port map( Selector => Selector, A =>
A, B => B, Y => Y);

process begin A <= "001"; B <= "010"; Selector <= '1'; wait for
100 ns;

Selector <= '0';
wait for 100 ns;

A <= "100";

```

```

B <= "101";
Selector <= '1';
wait for 100 ns;

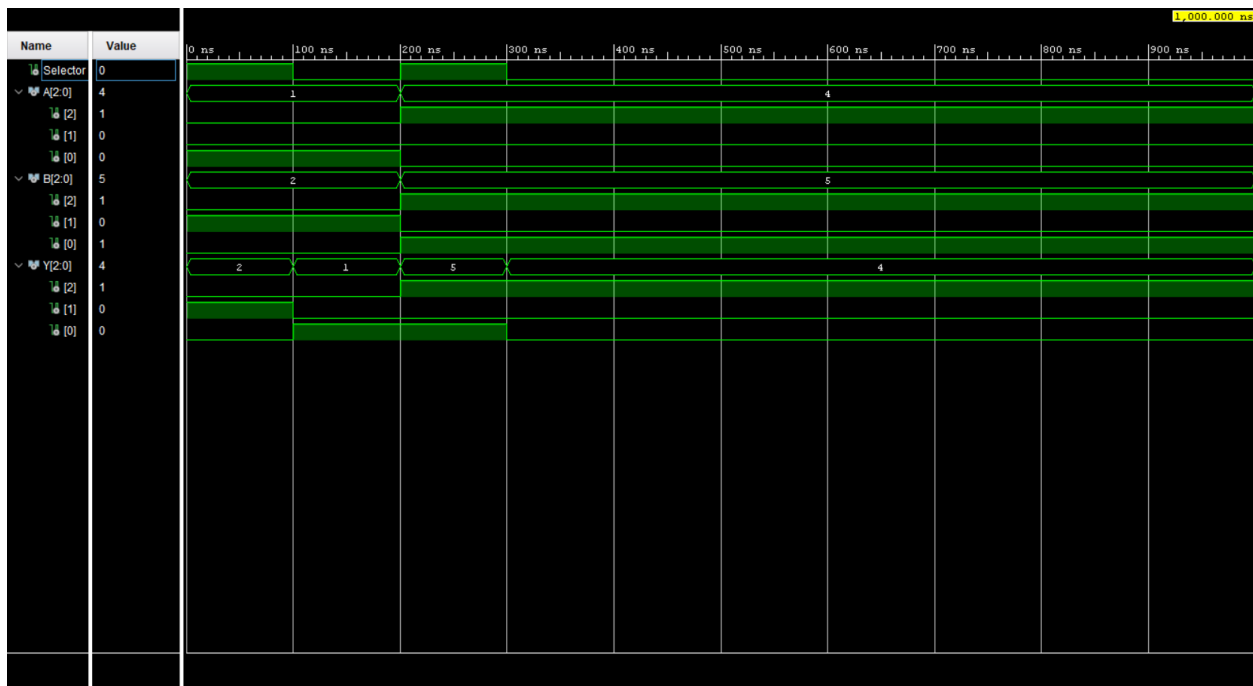
Selector <= '0';
wait;

end process;

end Behavioral;

```

Timing Diagram for 2 way 3 bit Multiplexer



6. 2 way 4 bit Multiplexer

Design Source VHDL code for 2 way 4 bit Multiplexer

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_2_way_4_bit is
    Port ( Selector : in STD_LOGIC;
          A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end entity;

```



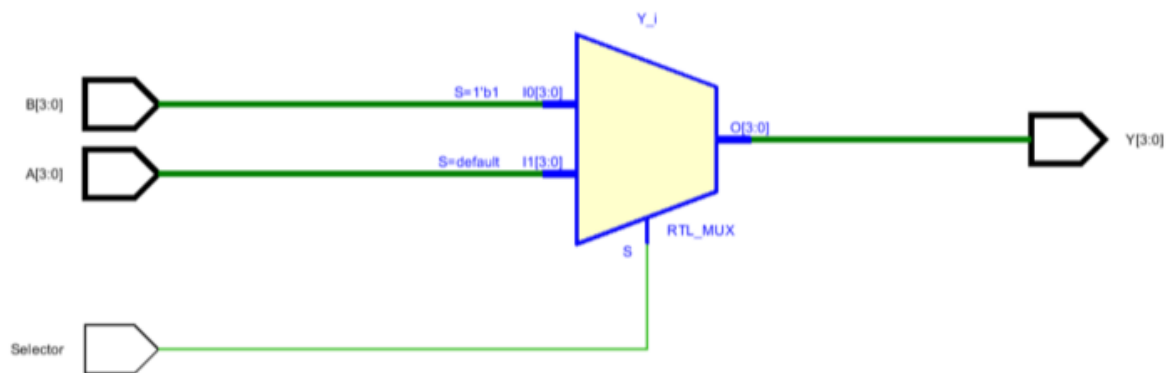
```
end MUX_2_way_4_bit;
```

```
architecture Behavioral of MUX_2_way_4_bit is
```

```
begin
```

```
    Y <= B when (Selector = '1') else A;
end Behavioral;
```

RTL Schematic diagram of 2 way 4 bit Multiplexer



Simulation source code for 2 way 4 bit Multiplexer

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_MUX_2_way_4_bit is
-- Port ( );
end TB_MUX_2_way_4_bit;

architecture Behavioral of TB_MUX_2_way_4_bit is
component MUX_2_way_4_bit
    PORT ( A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          Selector : in STD_LOGIC;
          Y : out STD_LOGIC_VECTOR(3 downto 0));
end component;

signal Selector : std_logic;
signal A,B,Y : std_logic_vector(3 downto 0);

begin
    UUT : MUX_2_way_4_bit
        port map(
```

```

        A => A,
        B => B,
        Selector => Selector,
        Y => Y);

process
begin
    A <= "0011";
    B <= "1000";
    Selector <= '1';
    wait for 100 ns;

    Selector <= '0';
    wait for 100 ns;

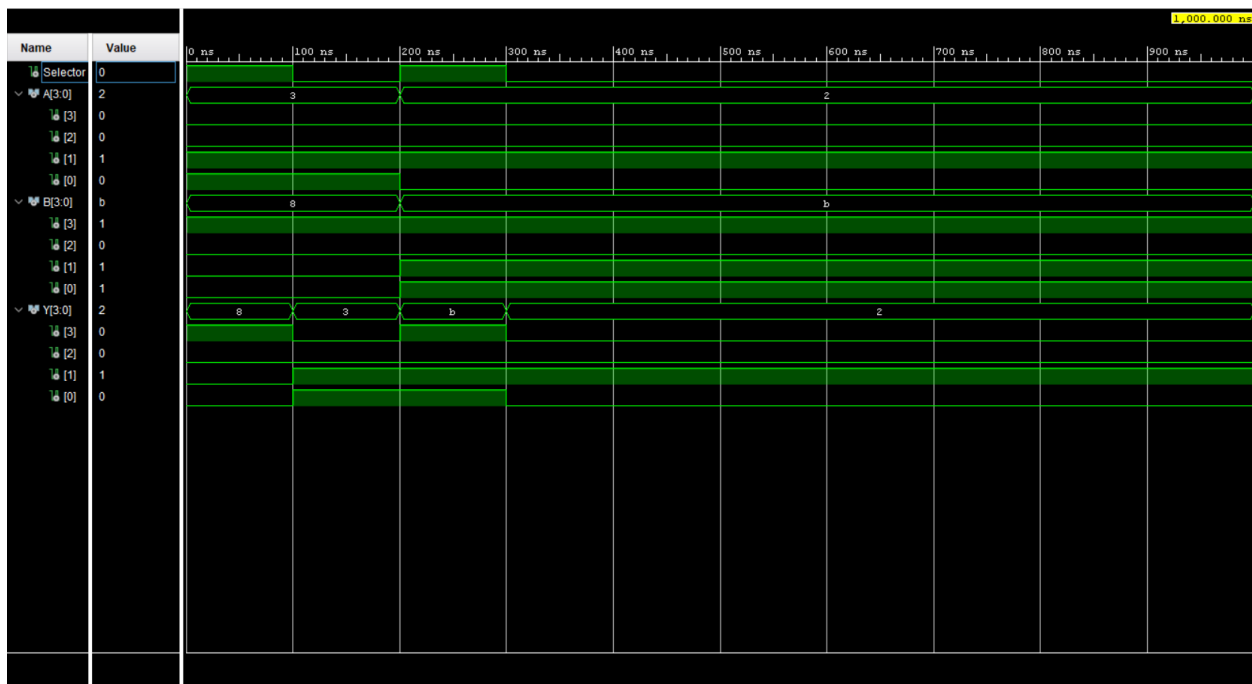
    A <= "0010";
    B <= "1011";
    Selector <= '1';
    wait for 100 ns;

    Selector <= '0';
    wait;
end process;

end Behavioral;

```

Timing Diagram for 2 way 4 bit Multiplexer



7. 8 way 4 bit Multiplexer

Design Source VHDL code for 8 way 4 bit Multiplexer

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX_8_way_4_bit is
    Port ( Selector : in STD_LOGIC_VECTOR (2 downto 0);
          A0 : in STD_LOGIC_VECTOR (3 downto 0);
          A1 : in STD_LOGIC_VECTOR (3 downto 0);
          A2 : in STD_LOGIC_VECTOR (3 downto 0);
          A3 : in STD_LOGIC_VECTOR (3 downto 0);
          A4 : in STD_LOGIC_VECTOR (3 downto 0);
          A5 : in STD_LOGIC_VECTOR (3 downto 0);
          A6 : in STD_LOGIC_VECTOR (3 downto 0);
          A7 : in STD_LOGIC_VECTOR (3 downto 0);
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end MUX_8_way_4_bit;

architecture Behavioral of MUX_8_way_4_bit is

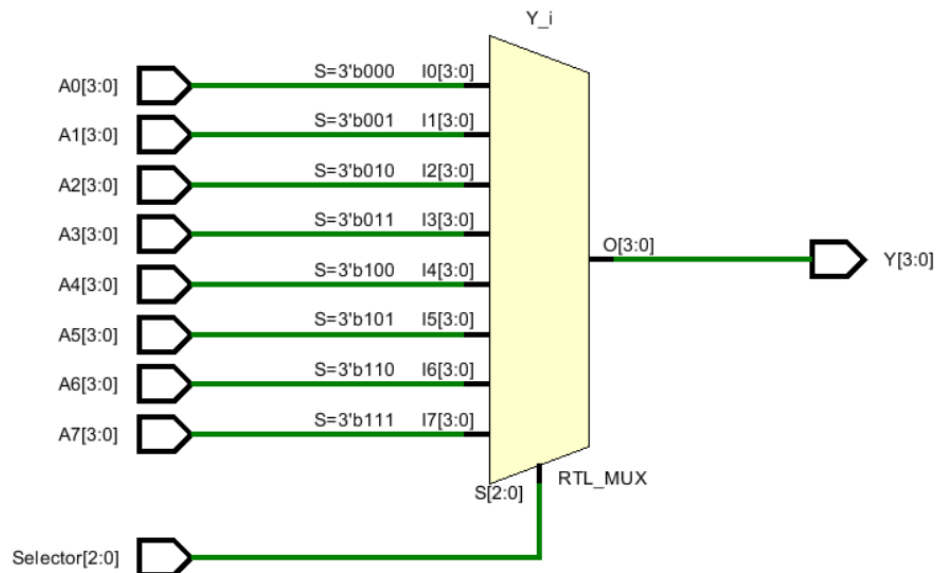
```

```

begin
  with Selector select
    Y <= A0 when "000",
        A1 when "001",
        A2 when "010",
        A3 when "011",
        A4 when "100",
        A5 when "101",
        A6 when "110",
        A7 when "111",
        (others => '0') when others;
end Behavioral;

```

RTL Schematic diagram of 8 way 4 bit Multiplexer



Simulation source code for 8 way 4 bit Multiplexer

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

```

```

entity TB_MUX_8_way_4_bit is
--  Port ( );
end TB_MUX_8_way_4_bit;

architecture Behavioral of TB_MUX_8_way_4_bit is
component MUX_8_way_4_bit
    Port( Selector: in STD_LOGIC_VECTOR(2 downto 0);
          A0 : in STD_LOGIC_VECTOR(3 downto 0);
          A1 : in STD_LOGIC_VECTOR(3 downto 0);
          A2 : in STD_LOGIC_VECTOR(3 downto 0);
          A3 : in STD_LOGIC_VECTOR(3 downto 0);
          A4 : in STD_LOGIC_VECTOR(3 downto 0);
          A5 : in STD_LOGIC_VECTOR(3 downto 0);
          A6 : in STD_LOGIC_VECTOR(3 downto 0);
          A7 : in STD_LOGIC_VECTOR(3 downto 0);
          Y : out STD_LOGIC_VECTOR(3 downto 0));
end component;

signal Selector : std_logic_vector(2 downto 0);
signal A0,A1,A2,A3,A4,A5,A6,A7,Y : std_logic_vector(3 downto 0);

begin
UUT : MUX_8_way_4_bit
    port map (
        Selector => Selector,
        A0 => A0,
        A1 => A1,
        A2 => A2,
        A3 => A3,
        A4 => A4,
        A5 => A5,
        A6 => A6,
        A7 => A7,
        Y => Y);

process
begin
    A0 <= "1111";
    A1 <= "0101";
    A2 <= "1110";
    A3 <= "1000";
    A4 <= "0000";
    A5 <= "0001";
    A6 <= "1001";

```

```
A7 <= "1010";

Selector <= "000";
wait for 100 ns;

Selector <= "001";
wait for 100 ns;

Selector <= "010";
wait for 100 ns;

Selector <= "011";
wait for 100 ns;

Selector <= "100";
wait for 100 ns;

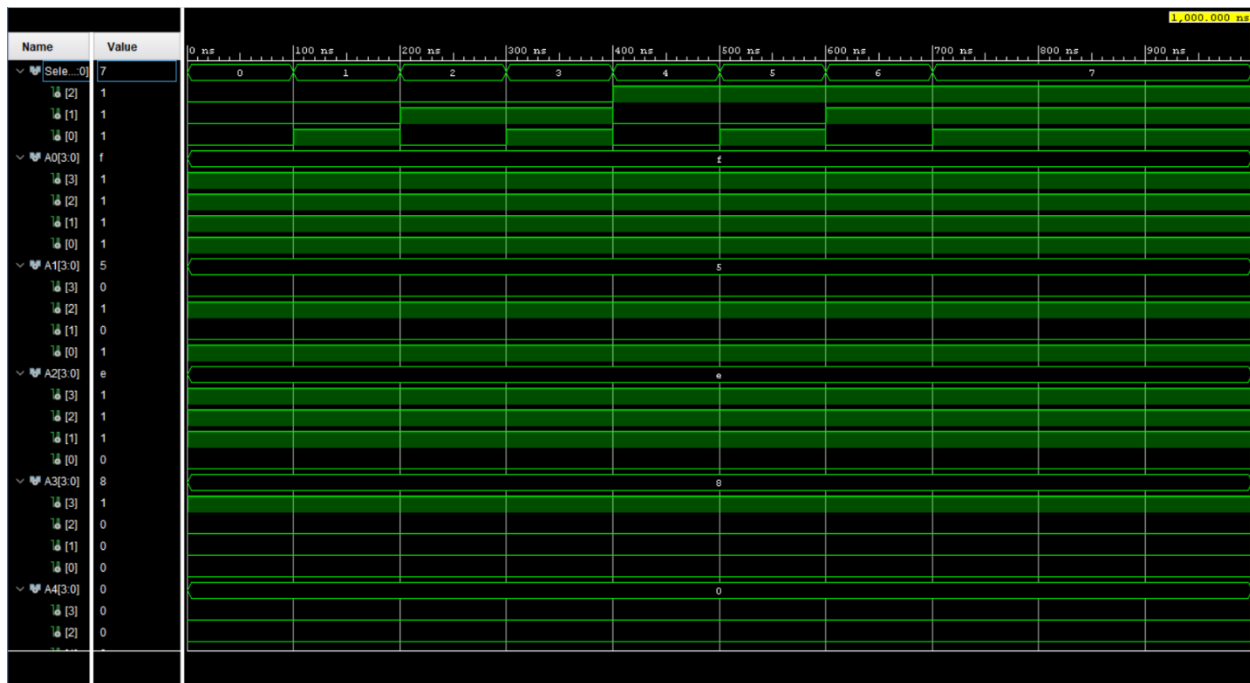
Selector <= "101";
wait for 100 ns;

Selector <= "110";
wait for 100 ns;

Selector <= "111";
wait;
end process;

end Behavioral;
```

Timing Diagram for 8 way 4 bit Multiplexer



8. Decoder _3_to_8

Design source VHDL code of Decoder _3 to 8

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Decoder_3_to_8 is
    Port (
        I   : in  STD_LOGIC_VECTOR (2 downto 0); -- 3-bit input
        EN  : in  STD_LOGIC;                      -- Enable
        Y   : out STD_LOGIC_VECTOR (7 downto 0)   -- 8-bit output
    );
end Decoder_3_to_8;

architecture Behavioral of Decoder_3_to_8 is
begin
    process(I, EN)
    begin
        Y <= (others => '0'); -- Default output (all zeros)
        if EN = '1' then
            case I is
                when "000" => Y(0) <= '1';
                when "001" => Y(1) <= '1';
                when "010" => Y(2) <= '1';
            end case;
        end if;
    end process;
end Behavioral;

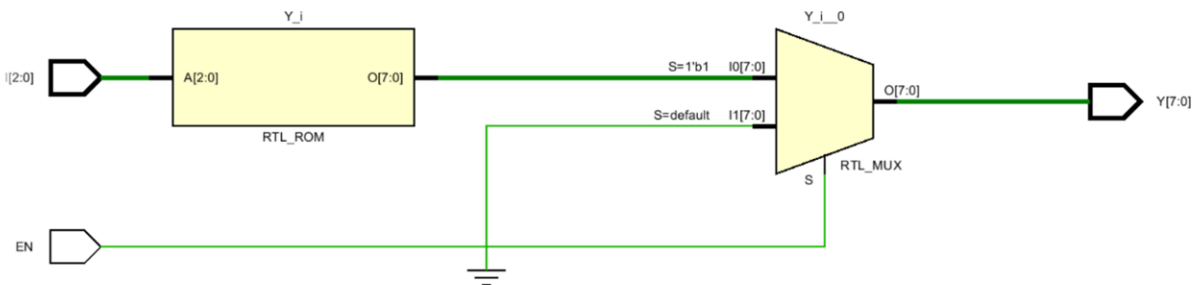
```

```

        when "011" => Y(3) <= '1';
        when "100" => Y(4) <= '1';
        when "101" => Y(5) <= '1';
        when "110" => Y(6) <= '1';
        when "111" => Y(7) <= '1';
        when others => null;
    end case;
end if;
end process;
end Behavioral;

```

RTL Schematic diagram of Decoder 3 to 8



Behavioral simulation Code for Decoder 3 to 8

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Decoder_3_to_8 is
end tb_Decoder_3_to_8;

architecture Behavioral of tb_Decoder_3_to_8 is

    component Decoder_3_to_8
    Port (
        I  : in  STD_LOGIC_VECTOR (2 downto 0);
        EN : in  STD_LOGIC;
        Y  : out STD_LOGIC_VECTOR (7 downto 0)
    );

```



```

end component;

signal I   : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal EN  : STD_LOGIC := '0';
signal Y   : STD_LOGIC_VECTOR(7 downto 0);

begin

    uut: Decoder_3_to_8 PORT MAP (
        I => I,
        EN => EN,
        Y => Y
    );

    stim_proc: process
    begin
        EN <= '0';
        I <= "000";
        wait for 20 ns;

        I <= "111";
        wait for 20 ns;

        EN <= '1';

        I <= "000";
        wait for 20 ns;

        I <= "001";
        wait for 20 ns;

        I <= "010";
        wait for 20 ns;

        I <= "011";
        wait for 20 ns;

        I <= "100";
        wait for 20 ns;

        I <= "101";
        wait for 20 ns;

        I <= "110";
        wait for 20 ns;
    end process;

```

```

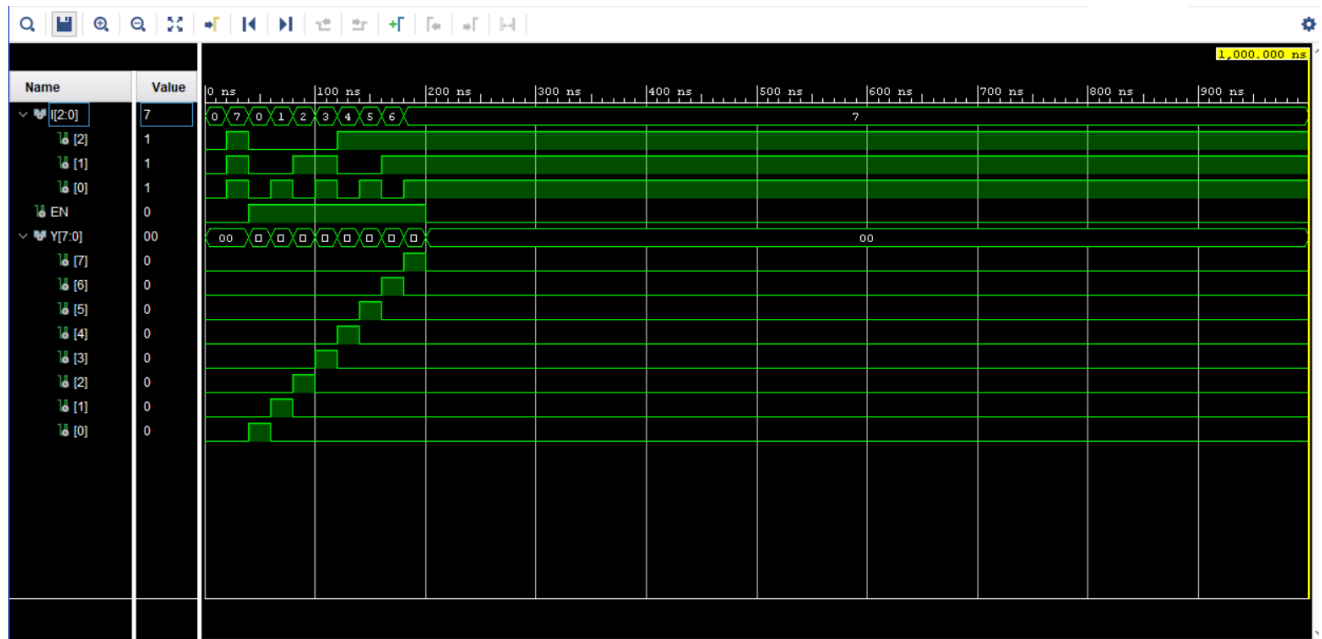
        I <= "111";
        wait for 20 ns;

        EN <= '0';
        wait;
    end process;

```

end Behavioral;

Timing Diagram for Decoder 3 to 8



9. Slow Clock

Design source VHDL code of Slow Clock

```

library IEEE;
use IEEE.STD LOGIC 1164.ALL;
use IEEE.NUMERIC STD.ALL;

entity Slow Clock is
    Port (
        Clk in    : in  STD LOGIC;
        Clk out   : out STD LOGIC
    );
end Slow Clock;

architecture Behavioral of Slow Clock is

```

```

    signal count : unsigned(25 downto 0) := (others => '0');
begin

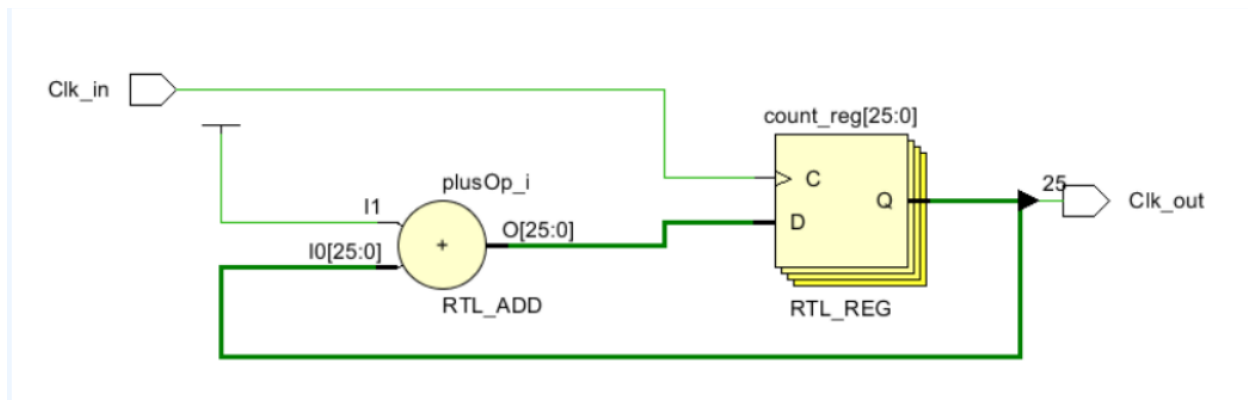
    process (Clk_in)
    begin
        if rising edge(Clk_in) then
            count <= count + 1;
        end if;
    end process;

    Clk_out <= count(25); -- Output a slower clock using MSB

end Behavioral;

```

RTL Schematic diagram of Slow Clock



10. D Flip Flop

Design source VHDL code of D Flip Flop

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FF is
    Port ( D : in STD_LOGIC;
          Res : in STD_LOGIC;
          Clk : in STD_LOGIC;
          Q : out STD_LOGIC);
end D_FF;

architecture Behavioral of D_FF is

begin

```

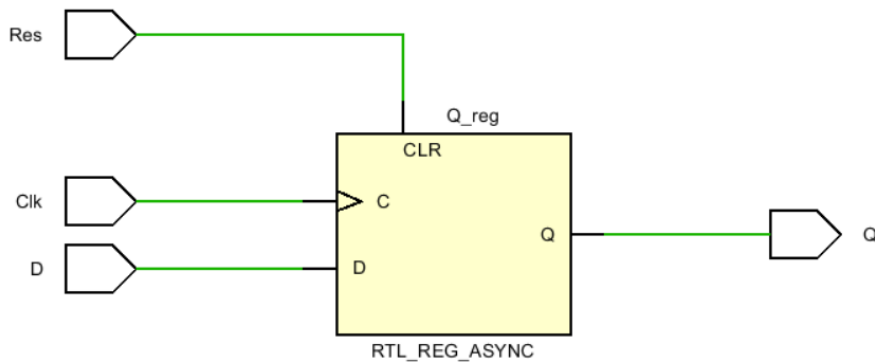
```

    process (Clk,Res)                                -- Process will be begin only
when Clk or Res signal changes
    begin
        if Res = '1' then
            Q <= '0';                                -- Resets to zero
        elsif (rising_edge(Clk)) then
            Q <= D;                                    -- Load the value of D to Q
        end if;
    end process;

end Behavioral;

```

RTL Schematic diagram of D Flip Flop



11. Register

Design source VHDL code of Register

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_4bit is
    Port (
        D      : in  STD_LOGIC_VECTOR (3 downto 0);
        En     : in  STD_LOGIC;
        Clk    : in  STD_LOGIC;
        Reset  : in  STD_LOGIC;
        Q      : out STD_LOGIC_VECTOR (3 downto 0)
    );
end Register_4bit;

architecture Behavioral of Register_4bit is
begin

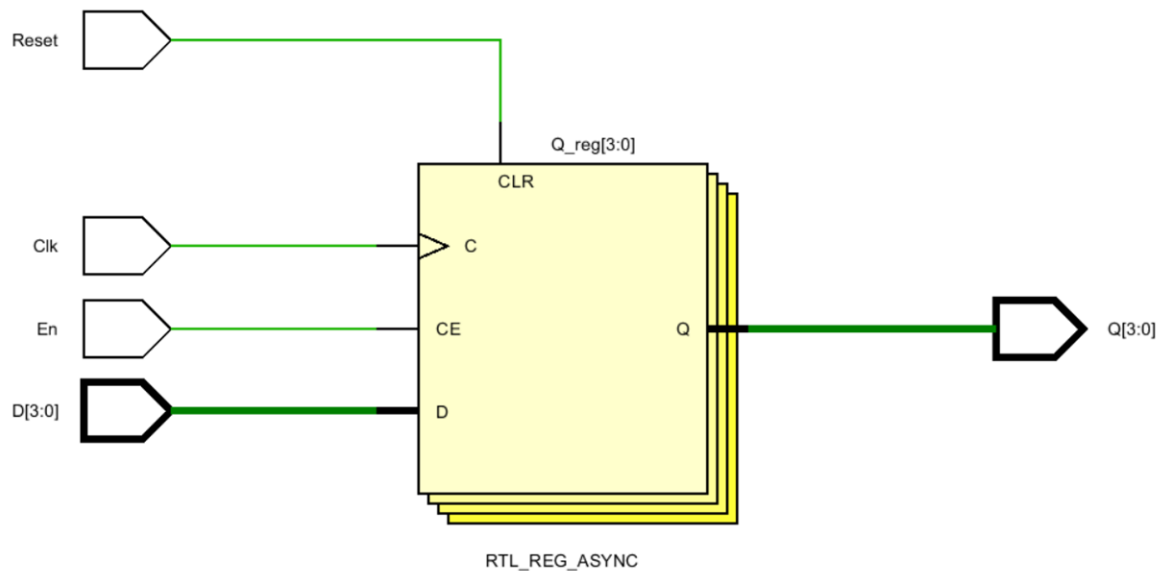
```

```

process(Clk, Reset)
begin
    if Reset = '1' then
        Q <= "0000"; -- Reset to 0
    elsif rising_edge(Clk) then
        if En = '1' then
            Q <= D;    -- Store input on clock edge
        end if;
    end if;
end process;
end Behavioral;

```

RTL Schematic diagram of 4bit Register



Behavioral simulation Code for 4bit Register

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Register_4bit is
end tb_Register_4bit;

architecture Behavioral of tb_Register_4bit is

    component Register_4bit
    Port (

```

```

        D      : in  STD_LOGIC_VECTOR (3 downto 0);
        En     : in  STD_LOGIC;
        Clk    : in  STD_LOGIC;
        Reset  : in  STD_LOGIC;
        Q      : out STD_LOGIC_VECTOR (3 downto 0)
    );
end component;

signal D      : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal En     : STD_LOGIC := '0';
signal Clk    : STD_LOGIC := '0';
signal Reset  : STD_LOGIC := '0';
signal Q      : STD_LOGIC_VECTOR(3 downto 0);

constant Clk_period : time := 10 ns;

begin

    uut: Register_4bit PORT MAP (
        D => D,
        En => En,
        Clk => Clk,
        Reset => Reset,
        Q => Q
    );

    Clk_process :process
    begin
        Clk <= '0';
        wait for Clk_period/2;
        Clk <= '1';
        wait for Clk_period/2;
    end process;

    stim_proc: process
    begin
        Reset <= '1';
        wait for 20 ns;

        Reset <= '0';
        wait for 10 ns;

        D <= "1010";
        En <= '0';
        wait for 20 ns;
    end process;

```

```

    En <= '1';
    wait for 20 ns;

    D <= "0101";
    wait for 20 ns;

    En <= '0';
    D <= "1111";
    wait for 20 ns;

    Reset <= '1';
    wait for 20 ns;

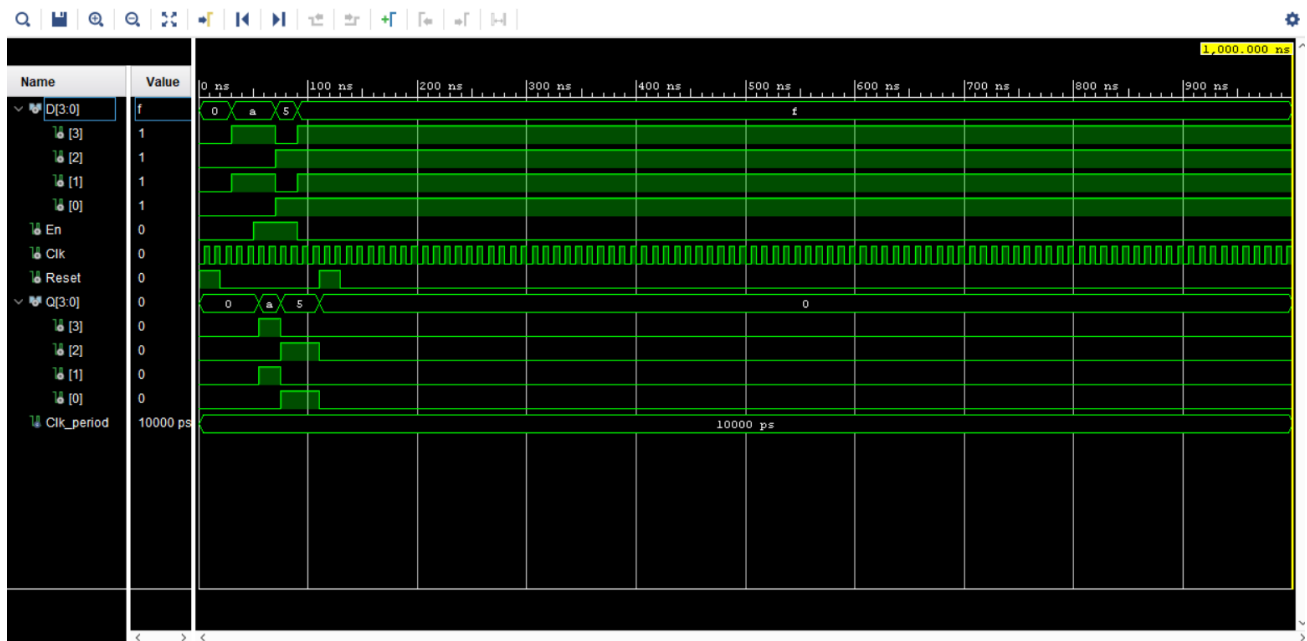
    Reset <= '0';

    wait;
end process;

end Behavioral;

```

Timing Diagram for 4 bit Register



12. Register Bank

Design source VHDL code of Register Bank

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register_Bank is
    Port ( Clk : in STD_LOGIC;
          Reset : in STD_LOGIC;
          D : in STD_LOGIC_VECTOR (3 downto 0);
          R0 : out STD_LOGIC_VECTOR (3 downto 0);
          R1 : out STD_LOGIC_VECTOR (3 downto 0);
          R2 : out STD_LOGIC_VECTOR (3 downto 0);
          R3 : out STD_LOGIC_VECTOR (3 downto 0);
          R4 : out STD_LOGIC_VECTOR (3 downto 0);
          R5 : out STD_LOGIC_VECTOR (3 downto 0);
          R6 : out STD_LOGIC_VECTOR (3 downto 0);
          R7 : out STD_LOGIC_VECTOR (3 downto 0);
          I : in STD_LOGIC_VECTOR (2 downto 0));
end Register_Bank;

architecture Behavioral of Register_Bank is

    component Decoder_3_to_8
        PORT( I : in STD_LOGIC_VECTOR (2 downto 0);
              EN : in STD_LOGIC;
              Y : out STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component Register_4bit
        PORT ( D : in STD_LOGIC_VECTOR (3 downto 0);
              En : in STD_LOGIC;
              Clk : in STD_LOGIC;
              Reset : in STD_LOGIC;
              Q : out STD_LOGIC_VECTOR (3 downto 0) );
    end component;

    signal EN,Reg_S : STD_LOGIC:='1';
    signal Y : STD_LOGIC_VECTOR(7 downto 0);

begin
    Decoder_3_to_8_new:Decoder_3_to_8
        PORT MAP (
```



```
I=>I,  
EN=>EN,  
Y=>Y  
);
```

```
Register_4bit_0 : Register_4bit  
PORT MAP(  
    D => "0000",  
    En => Y(0),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R0  
);
```

```
Register_4bit_1 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(1),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R1  
);
```

```
Register_4bit_2 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(2),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R2  
);
```

```
Register_4bit_3 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(3),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R3  
);
```

```
Register_4bit_4 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(4),  
    Clk => Clk,  
    Reset => Reg_S,
```

```
Q => R4  
);
```

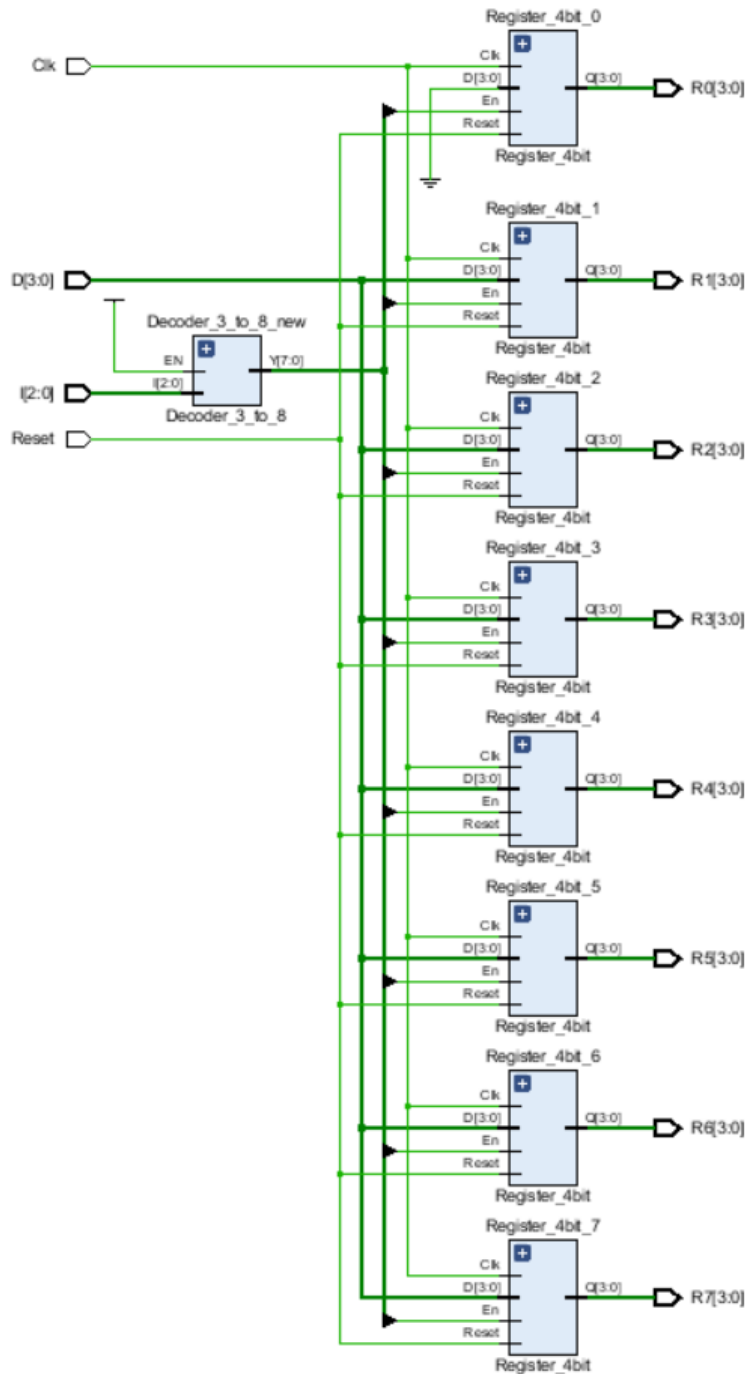
```
Register_4bit_5 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(5),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R5  
);
```

```
Register_4bit_6 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(6),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R6  
);
```

```
Register_4bit_7 : Register_4bit  
PORT MAP(  
    D => D,  
    En => Y(7),  
    Clk => Clk,  
    Reset => Reg_S,  
    Q => R7  
);
```

```
Reg_S <= Reset;  
end Behavioral;
```

RTL Schematic diagram of Register Bank



Behavioral simulation Code for Register Bank

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Register_Bank is
```

```

-- A testbench entity is always empty
end tb_Register_Bank;

architecture Behavioral of tb_Register_Bank is

    -- 1. Component Declaration for the Unit Under Test (UUT)
    component Register_Bank
    Port (
        Clk      : in STD_LOGIC;
        Reset    : in STD_LOGIC;
        D        : in STD_LOGIC_VECTOR (3 downto 0);
        R0       : out STD_LOGIC_VECTOR (3 downto 0);
        R1       : out STD_LOGIC_VECTOR (3 downto 0);
        R2       : out STD_LOGIC_VECTOR (3 downto 0);
        R3       : out STD_LOGIC_VECTOR (3 downto 0);
        R4       : out STD_LOGIC_VECTOR (3 downto 0);
        R5       : out STD_LOGIC_VECTOR (3 downto 0);
        R6       : out STD_LOGIC_VECTOR (3 downto 0);
        R7       : out STD_LOGIC_VECTOR (3 downto 0);
        I        : in STD_LOGIC_VECTOR (2 downto 0)
    );
    end component;

    -- 2. Declare internal signals to connect to the UUT
    -- Inputs (initialized to zero)
    signal Clk      : std_logic := '0';
    signal Reset    : std_logic := '0';
    signal D        : std_logic_vector(3 downto 0) := (others => '0');
    signal I        : std_logic_vector(2 downto 0) := (others => '0');

    -- Outputs
    signal R0 : std_logic_vector(3 downto 0);
    signal R1 : std_logic_vector(3 downto 0);
    signal R2 : std_logic_vector(3 downto 0);
    signal R3 : std_logic_vector(3 downto 0);
    signal R4 : std_logic_vector(3 downto 0);
    signal R5 : std_logic_vector(3 downto 0);
    signal R6 : std_logic_vector(3 downto 0);
    signal R7 : std_logic_vector(3 downto 0);

    -- Define the Clock period
    constant Clk_period : time := 10 ns;

begin

```

```

-- 3. Instantiate the Unit Under Test (UUT)
 uut: Register_Bank PORT MAP (
     Clk => Clk,
     Reset => Reset,
     D => D,
     R0 => R0,
     R1 => R1,
     R2 => R2,
     R3 => R3,
     R4 => R4,
     R5 => R5,
     R6 => R6,
     R7 => R7,
     I => I
 );

-- 4. Clock Generation Process
 Clk_process :process
begin
    Clk <= '0';
    wait for Clk_period/2;
    Clk <= '1';
    wait for Clk_period/2;
end process;

-- 5. Stimulus Process (Applying test inputs)
 stim_proc: process
begin
    -- Step A: Apply Reset to clear all registers
    Reset <= '1';
    wait for 20 ns;
    Reset <= '0';
    wait for Clk_period;

    -- Step B: Write Data to Register 1 (R1)
    D <= "1010"; -- Hex A
    I <= "001"; -- Address for R1
    wait for Clk_period;

    -- Step C: Write Data to Register 2 (R2)
    D <= "1100"; -- Hex C
    I <= "010"; -- Address for R2
    wait for Clk_period;

    -- Step D: Write Data to Register 7 (R7)

```

```

    D <= "0111";  -- Hex 7
    I <= "111";   -- Address for R7
    wait for Clk_period;

    -- Step E: Try writing to Register 0 (R0)
    -- *Note: Because your source code maps R0's data input
directly to "0000",
    -- it will act as a constant zero register, which is common in
CPU design.
    -- Even though D="1111", R0 should remain "0000" in the
simulation.
    D <= "1111";  -- Hex F
    I <= "000";   -- Address for R0
    wait for Clk_period;

    -- Step F: Write Data to Register 3 (R3)
    D <= "0011";  -- Hex 3
    I <= "011";   -- Address for R3
    wait for Clk_period;

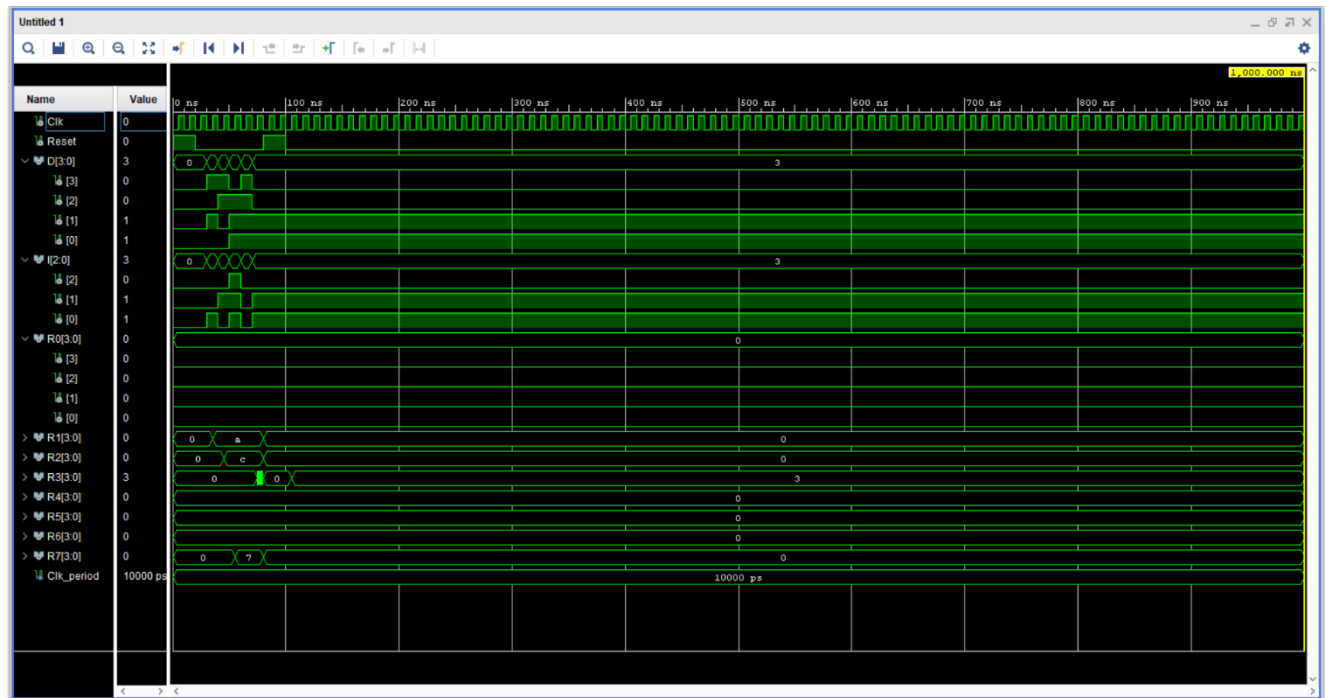
    -- Step G: Apply Reset again to verify all registers clear
correctly
    Reset <= '1';
    wait for 20 ns;
    Reset <= '0';

    -- End the simulation
    wait;
end process;

end Behavioral;

```

Timing Diagram for Register Bank



13. Rom

Design source VHDL code of Rom

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity Program_ROM is
    Port ( address : in STD_LOGIC_VECTOR (2 downto 0);
          instruction_code : out STD_LOGIC_VECTOR (11 downto 0));
end Program_ROM;

architecture Behavioral of Program_ROM is

    type rom_type is array (0 to 7) of std_logic_vector(11 downto 0);

    signal twelveSegment_ROM : rom_type := ( -- Program is hardcoded
here.

-- TASK 01 : Instructions as in Moodle

-- "101110001010", --0 Movi R7, 10
```

```

--      "100100000001", --1 Movi R2, 1
--      "010100000000", --2 Neg  R2
--      "001110100000", --3 Add  R7, R2
--      "111110000111", --4 JZR R7, 7
--      "110000000011", --5 JZR R0, 3
--      "000000000000", --6
--      "000000000000"  --7

-- TASK 02 : Write an Assembly program to calculate the total of all
integers between 1 and 3

      "100010000001", --0 Movi R1, 1
      "100100000010", --1 Movi R2, 2
      "001110010000", --2 Add  R7, R1
      "001110100000", --3 Add  R7, R2
      "100110000011", --4 Movi R3, 3
      "001110110000", --5 Add  R7, R3
      "100010000000", --6 Movi R1, 0
      "000000000000"  --7

);

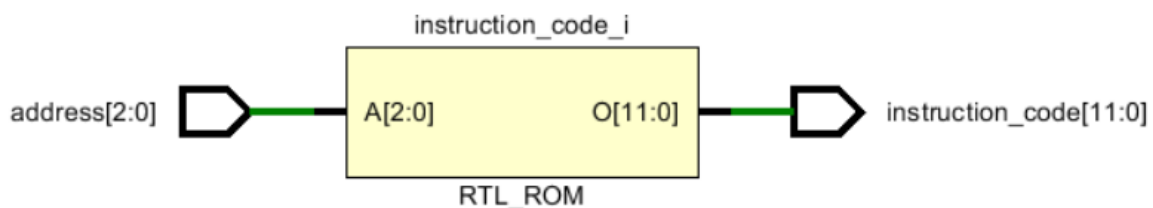
begin

instruction_code <= twelveSegmant_ROM(to_integer(unsigned(address)));

end Behavioral;

```

RTL Schematic diagram of Rom



Behavioral simulation Code for ROM

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```



```

entity sim_ROM is
end sim_ROM;

architecture Behavioral of sim_ROM is

-- Testbench signals
signal Memory_Sel_tb : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Instruction_bus_tb : STD_LOGIC_VECTOR(11 downto 0);

begin

-- Direct entity instantiation (recommended)
UUT: entity work.Program_ROM
    port map (
        address => Memory_Sel_tb,
        instruction_code => Instruction_bus_tb
    );

-- 240107B---011 101 010 101 111 011
process
begin
    Memory_Sel_tb <= "000";
    wait for 100 ns;

    Memory_Sel_tb <= "001";
    wait for 100 ns;

    Memory_Sel_tb <= "010";
    wait for 100 ns;

    Memory_Sel_tb <= "011";
    wait for 100 ns;

    Memory_Sel_tb <= "100";
    wait for 100 ns;

    Memory_Sel_tb <= "101";
    wait for 100 ns;

    Memory_Sel_tb <= "110";
    wait for 100 ns;
end process;

```

```

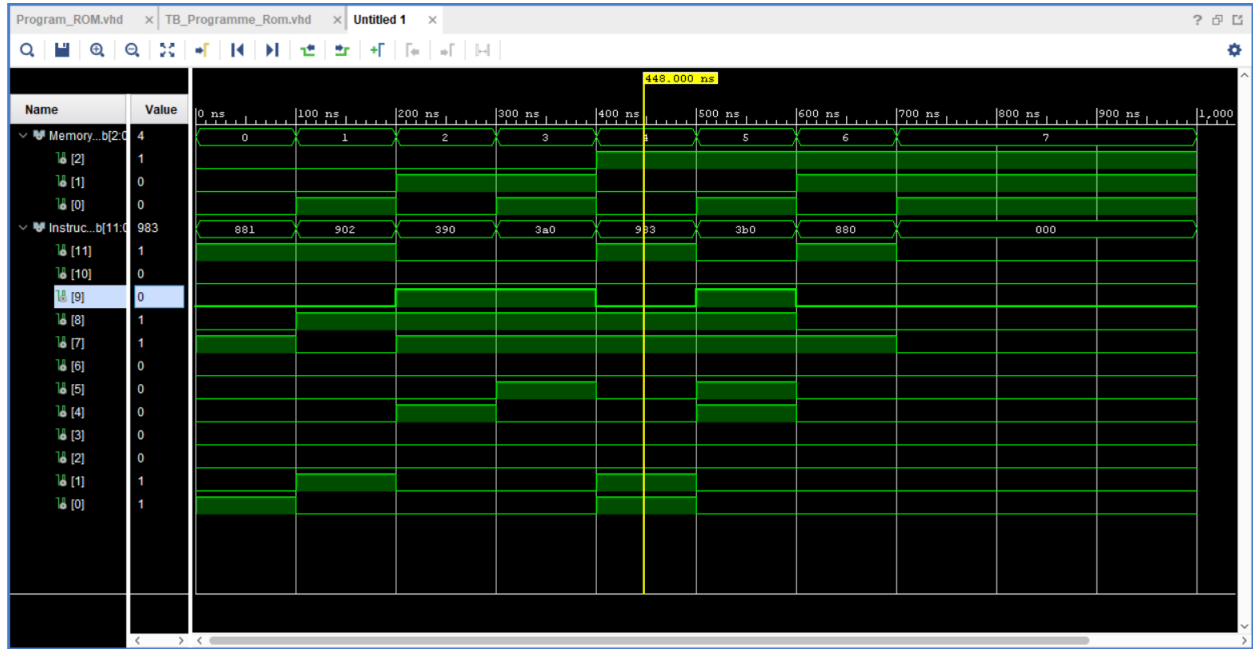
        Memory_Sel_tb <= "111";
        wait for 100 ns;

        wait;
end process;

end Behavioral;

```

Timing Diagram for ROM



14. Program Counter

Design source VHDL code of Program Counter

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity PC_3_bit is
    Port ( D : in STD_LOGIC_VECTOR (2 downto 0);
          Res : in STD_LOGIC;
          Clk : in STD_LOGIC;

```

```

        Q : out STD_LOGIC_VECTOR (2 downto 0));
end PC_3_bit;

architecture Behavioral of PC_3_bit is
    component D_FF is
        Port ( D : in STD_LOGIC;
              Res : in STD_LOGIC;
              Clk : in STD_LOGIC;
              Q : out STD_LOGIC);
    end component;

    signal Res_Sig : STD_LOGIC := '1';

begin

    DFF_0 : D_FF
        PORT MAP (D => D(0), Q => Q(0), Clk => Clk, Res => Res_Sig);

    DFF_1 : D_FF
        PORT MAP (D => D(1), Q => Q(1), Clk => Clk, Res => Res_Sig);

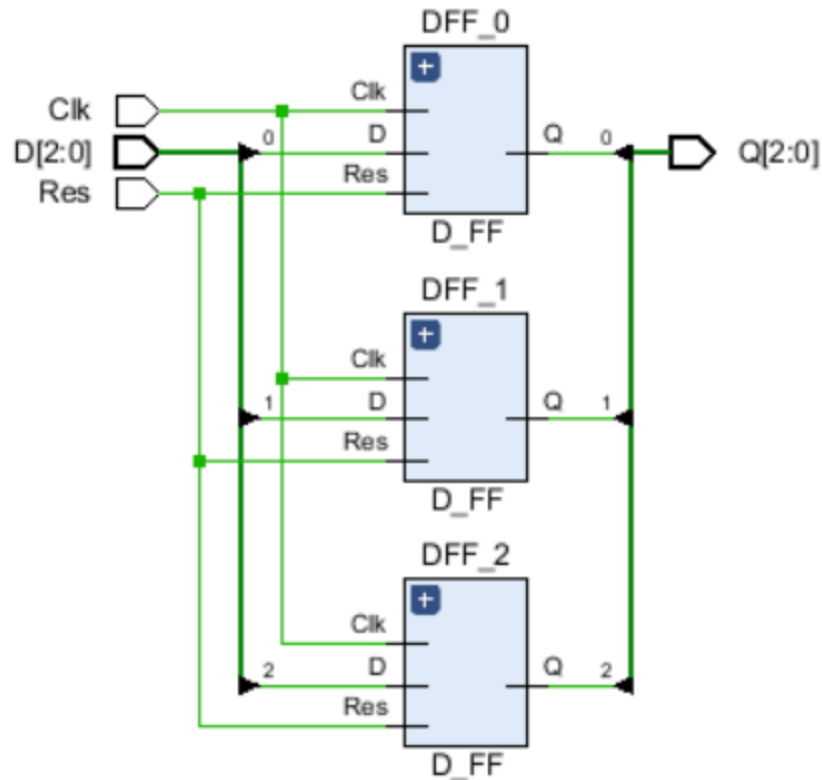
    DFF_2 : D_FF
        PORT MAP (D => D(2), Q => Q(2), Clk => Clk, Res => Res_Sig);

    Res_Sig <= Res;

end Behavioral;

```

RTL Schematic diagram of Program Counter



Behavioral simulation Code for Program Counter

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PC_3_bit_tb is
end PC_3_bit_tb;

architecture Behavioral of PC_3_bit_tb is

    -- Component Declaration
    component PC_3_bit is
        Port ( D      : in STD_LOGIC_VECTOR (2 downto 0);
              Res    : in STD_LOGIC;
              Clk     : in STD_LOGIC;
              Q       : out STD_LOGIC_VECTOR (2 downto 0));
    end component;

    -- Signals
    signal D      : STD_LOGIC_VECTOR (2 downto 0) := "000";
    signal Res    : STD_LOGIC := '0';
    signal Clk    : STD_LOGIC := '0';
    signal Q      : STD_LOGIC_VECTOR (2 downto 0);


```

```

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: PC_3_bit
        PORT MAP (
            D    => D,
            Res  => Res,
            Clk  => Clk,
            Q    => Q
        );

    -- Clock Process (10 ns period)
    clk_process : process
    begin
        while true loop
            Clk <= '0';
            wait for 5 ns;
            Clk <= '1';
            wait for 5 ns;
        end loop;
    end process;

    -- Stimulus Process
    stim_proc: process
    begin

        -- Apply Reset
        Res <= '1';
        wait for 10 ns;
        Res <= '0';

        -- Apply inputs
        D <= "001"; wait for 10 ns;
        D <= "010"; wait for 10 ns;
        D <= "011"; wait for 10 ns;
        D <= "100"; wait for 10 ns;
        D <= "101"; wait for 10 ns;

        -- Test reset again
        Res <= '1'; wait for 10 ns;
        Res <= '0';

        D <= "111"; wait for 10 ns;

        -- End simulation

```

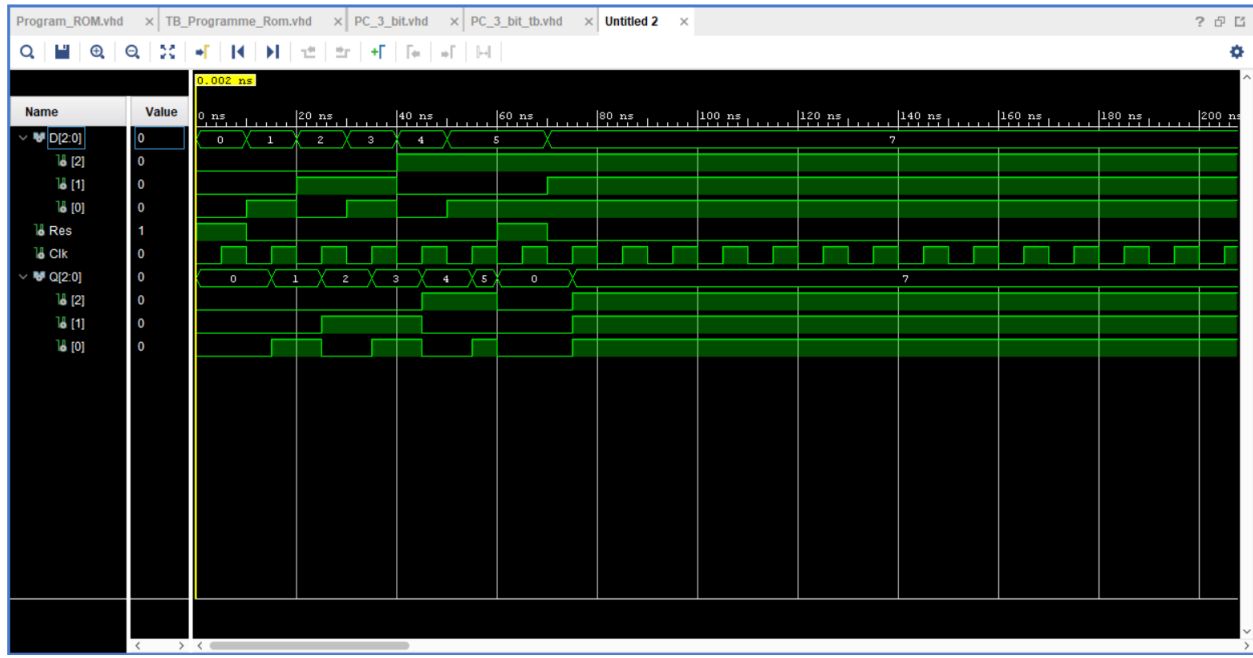
```

        wait;
    end process;

end Behavioral;

```

Timing Diagram for Program Counter



15. Add_Sub_Unit

Design source VHDL code of Add SUB unit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ADD_SUB_4bit is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);      -- A and B 4
    bit inputs
        Operation : in STD_LOGIC;                  -- ADD - '0'
    and SUB - '1'
        C_out : out STD_LOGIC;
        S : out STD_LOGIC_VECTOR (3 downto 0);      -- S 4 bit
    output
        Overflow : out STD_LOGIC;
        Zero : out STD_LOGIC);                      -- '1', if the
    total result is zero
end ADD_SUB_4bit;

```

architecture Behavioral of ADD_SUB_4bit is

component RCA_4

```
    PORT(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          C_in : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR(3 downto 0);
          C_out : out STD_LOGIC);
```

end component;

SIGNAL Y,X : STD_LOGIC_VECTOR(3 downto 0);

begin

```
X(0) <= B(0) XOR Operation;
X(1) <= B(1) XOR Operation;
X(2) <= B(2) XOR Operation;
X(3) <= B(3) XOR Operation;
```

RCA0 : RCA_4

```
    port map(
        A => A,
        B => X,
        C_in => Operation,
        S => Y,
        C_out => C_out);
```

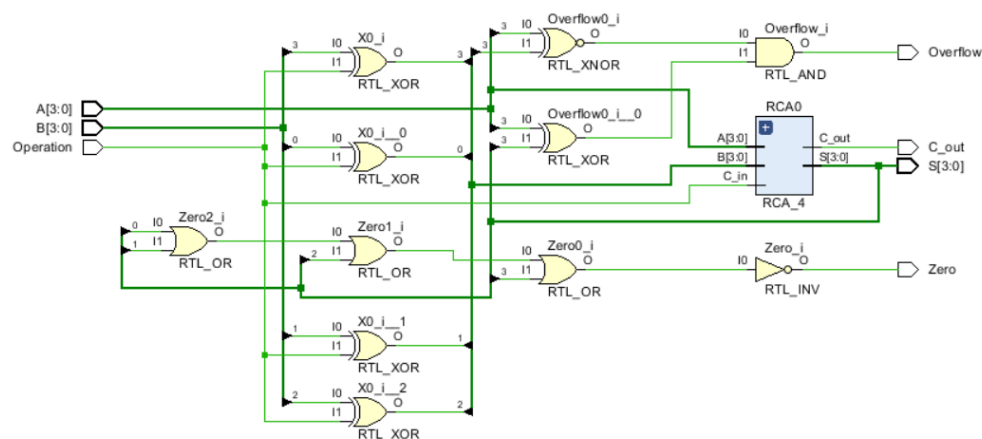
S <= Y;

Overflow <= (A(3) XNOR (Operation XOR B(3))) AND (A(3) XOR Y(3));

Zero <= NOT(Y(0) OR Y(1) OR Y(2) OR Y(3));

end Behavioral;

RTL Schematic diagram of Add SUB unit



Behavioral simulation Code for Add SUB unit

```
-- Company:
-- Engineer:
--
-- Create Date: 04/30/2026 10:56:42 AM
-- Design Name:
-- Module Name: TB_ADD_SUB - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
```

```
library IEEE;
use IEEE.STD LOGIC 1164.ALL;
```



```
entity TB_ADD_SUB is
end TB_ADD_SUB;
```

```
architecture Behavioral of TB_ADD_SUB is
```

```
    component ADD_SUB_4bit
        Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
              B : in STD_LOGIC_VECTOR (3 downto 0);
              Operation : in STD_LOGIC;
              C_out : out STD_LOGIC;
              S : out STD_LOGIC_VECTOR (3 downto 0);
              Overflow : out STD_LOGIC;
              Zero : out STD_LOGIC);
    end component;
```

```
-- Inputs
```

```
signal A : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal B : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal Operation : STD_LOGIC := '0';
```

```
-- Outputs
```

```
signal C_out : STD_LOGIC;
signal S : STD_LOGIC_VECTOR(3 downto 0);
signal Overflow : STD_LOGIC;
signal Zero : STD_LOGIC;
```

```
begin
```

```
    UUT: ADD_SUB_4bit port map (
        A => A,
        B => B,
        Operation => Operation,
        C_out => C_out,
        S => S,
        Overflow => Overflow,
        Zero => Zero
    );
```

```
    stim_proc: process
```

```
    begin
```

```
        -- ---- ADDITION TESTS (Operation = '0') ----
        Operation <= '0';
```

```
        -- 2 + 3 = 5
```

```

A <= "0010"; B <= "0011";
wait for 20 ns;

-- 7 + 1 = 8 (Signed Overflow Expected: Positive + Positive =
Negative)
A <= "0111"; B <= "0001";
wait for 20 ns;

-- ---- SUBTRACTION TESTS (Operation = '1') ----
Operation <= '1';

-- 5 - 2 = 3
A <= "0101"; B <= "0010";
wait for 20 ns;

-- 4 - 4 = 0 (Zero Flag Expected)
A <= "0100"; B <= "0100";
wait for 20 ns;

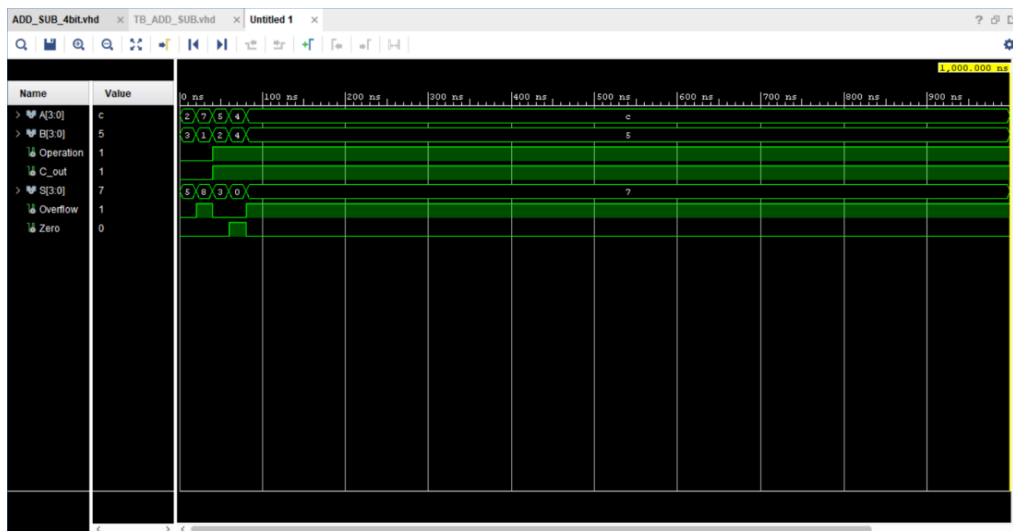
-- -4 - 5 = -9 (Signed Overflow Expected)
A <= "1100"; B <= "0101";
wait for 20 ns;

wait;
end process;

end Behavioral;

```

Timing Diagram for Add SUB unit



16. Instruction Decoder

Design source VHDL code of Instruction Decoder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Instruction_Decoder is
    Port (
        I : in STD_LOGIC_VECTOR (11 downto 0); -- 12-bit instruction
        Reg_Check_Jump : in STD_LOGIC_VECTOR (3 downto 0);
        Load_Select : out STD_LOGIC;
        Imm_Value : out STD_LOGIC_VECTOR (3 downto 0);
        Reg_Enable : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_1 : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_2 : out STD_LOGIC_VECTOR (2 downto 0);
        Add_Sub : out STD_LOGIC;
        Jump_Flag : out STD_LOGIC;
        Address : out STD_LOGIC_VECTOR (2 downto 0)
    );
end Instruction_Decoder;

architecture Behavioral of Instruction_Decoder is

    signal OpCode : STD_LOGIC_VECTOR(1 downto 0);
    signal RegA : STD_LOGIC_VECTOR(2 downto 0);
    signal RegB : STD_LOGIC_VECTOR(2 downto 0);
    signal Imm : STD_LOGIC_VECTOR(3 downto 0);

begin

    OpCode <= I(11 downto 10); --used for operations
    RegA <= I(9 downto 7);
    RegB <= I(6 downto 4);
    Imm <= I(3 downto 0);

    Reg_Enable <= RegA;
    Imm_Value <= Imm;
    Address <= I(2 downto 0);

    Load_Select <= '1' when (OpCode = "10") else '0';
    Add_Sub <= '1' when (OpCode = "01") else '0';

    process(OpCode, RegB, RegA)
    begin
        if OpCode = "01" then
            Reg_Select_1 <= "000";
        end if
    end process
end architecture;
```

```

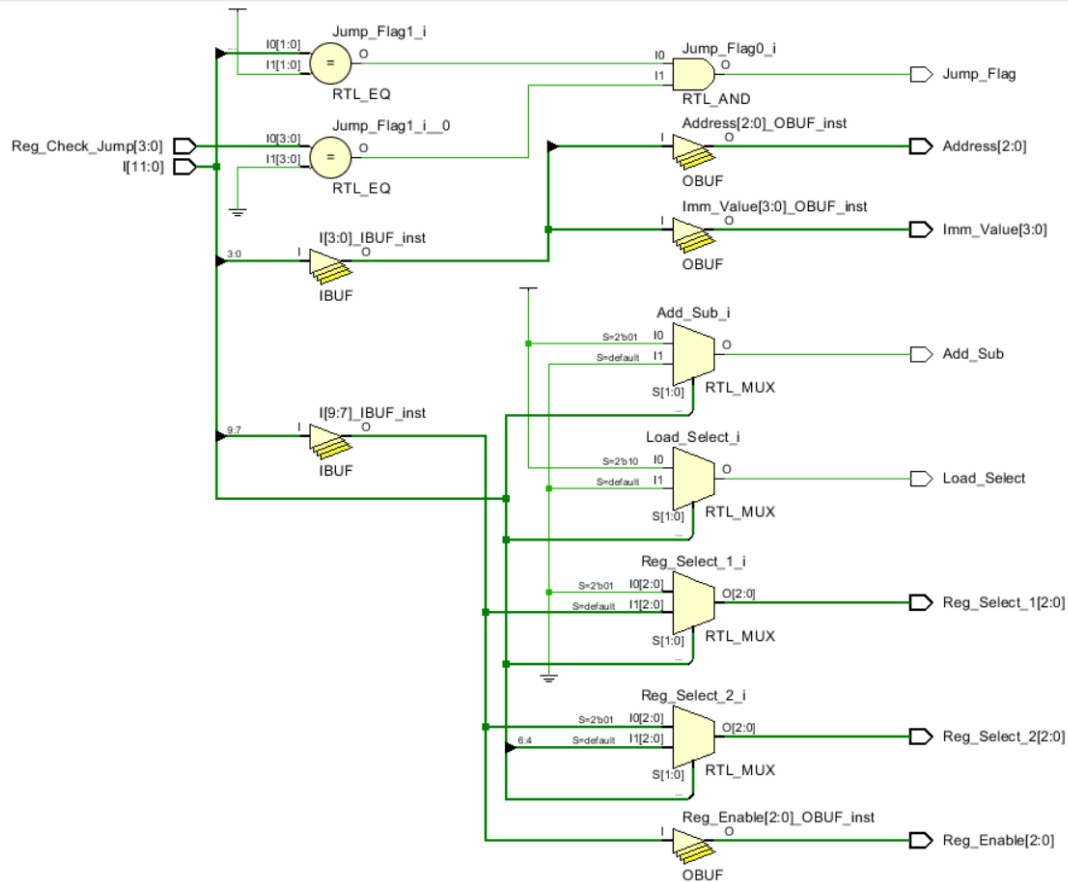
        Reg_Select_2 <= RegA;
    else
        Reg_Select_2 <= RegB;
        Reg_Select_1 <= RegA;
    end if;
end process;

Jump_Flag <= '1' when (OpCode = "11" and Reg_Check_Jump = "0000")
else '0';

end Behavioral;

```

RTL Schematic diagram of Instruction Decoder



Behavioral simulation Code for Instruction Decoder

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```
entity tb_Instruction_Decoder is
end tb_Instruction_Decoder;
```

architecture Behavioral of tb_Instruction_Decoder is

```
    component Instruction_Decoder
    Port (
        I                : in STD_LOGIC_VECTOR (11 downto 0);
        Reg_Check_Jump   : in STD_LOGIC_VECTOR (3 downto 0);
        Load_Select      : out STD_LOGIC;
        Imm_Value        : out STD_LOGIC_VECTOR (3 downto 0);
        Reg_Enable       : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_1     : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_2     : out STD_LOGIC_VECTOR (2 downto 0);
        Add_Sub          : out STD_LOGIC;
        Jump_Flag        : out STD_LOGIC;
        Address          : out STD_LOGIC_VECTOR (2 downto 0)
    );
    end component;

    signal I                : STD_LOGIC_VECTOR(11 downto 0) := (others
=> '0');
    signal Reg_Check_Jump : STD_LOGIC_VECTOR(3 downto 0) := (others =>
'0');

    signal Load_Select      : STD_LOGIC;
    signal Imm_Value        : STD_LOGIC_VECTOR(3 downto 0);
    signal Reg_Enable       : STD_LOGIC_VECTOR(2 downto 0);
    signal Reg_Select_1     : STD_LOGIC_VECTOR(2 downto 0);
    signal Reg_Select_2     : STD_LOGIC_VECTOR(2 downto 0);
    signal Add_Sub          : STD_LOGIC;
    signal Jump_Flag        : STD_LOGIC;
    signal Address          : STD_LOGIC_VECTOR(2 downto 0);

begin

    uut: Instruction_Decoder PORT MAP (
        I => I,
        Reg_Check_Jump => Reg_Check_Jump,
        Load_Select => Load_Select,
        Imm_Value => Imm_Value,
        Reg_Enable => Reg_Enable,
        Reg_Select_1 => Reg_Select_1,
        Reg_Select_2 => Reg_Select_2,
        Add_Sub => Add_Sub,
```

```

        Jump_Flag => Jump_Flag,
        Address => Address
    );

stim_proc: process
begin
    wait for 10 ns;

    I <= "000010100000";
    Reg_Check_Jump <= "0000";
    wait for 20 ns;

    I <= "010110000000";
    wait for 20 ns;

    I <= "101100001010";
    wait for 20 ns;

    I <= "111110000101";

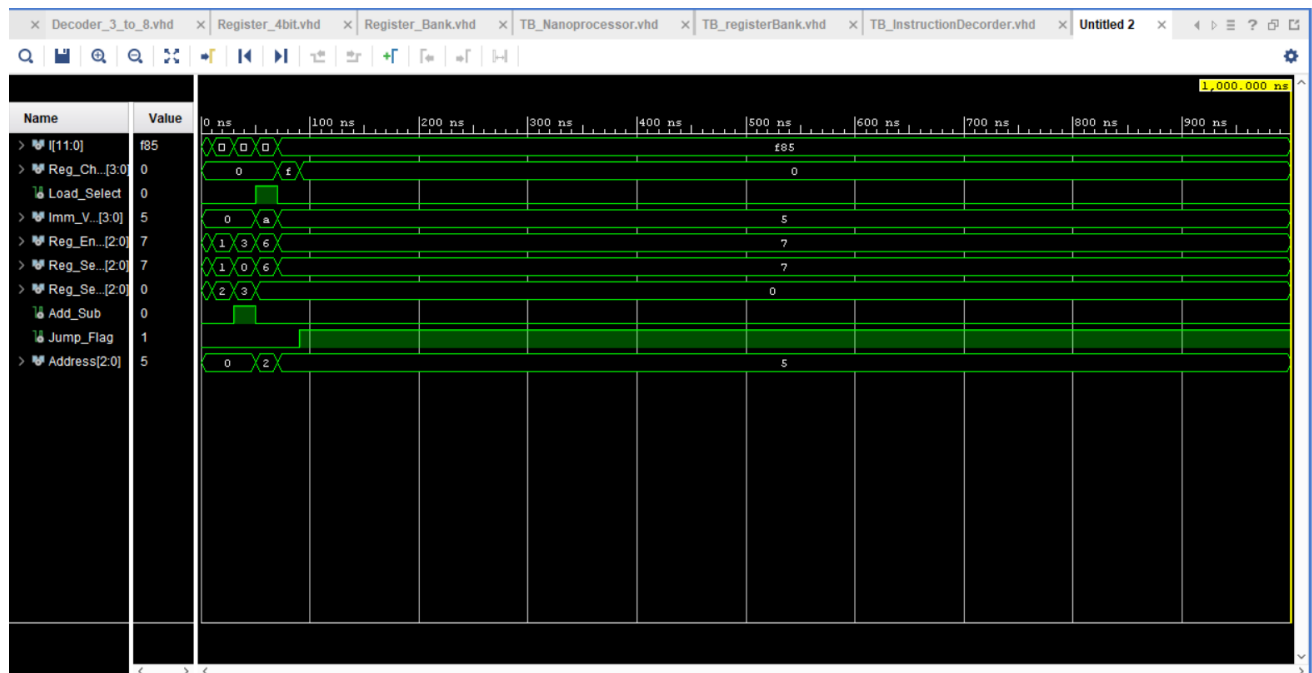
    Reg_Check_Jump <= "1111";
    wait for 20 ns;

    Reg_Check_Jump <= "0000";
    wait for 20 ns;

    wait;
end process;

end Behavioral;
```

Timing Diagram for Instruction Decoder



17. Adder_3bit

Design source VHDL code of Adder 3bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Adder_3bit is
    Port ( A : in STD_LOGIC_VECTOR (2 downto 0);
          S : out STD_LOGIC_VECTOR (2 downto 0);
          C_out : out STD_LOGIC);
end Adder_3bit;

architecture Behavioral of Adder_3bit is
    component FA
        Port (
            A      : in  STD_LOGIC;
            B      : in  STD_LOGIC;
            C_in   : in  STD_LOGIC;
            S      : out STD_LOGIC;
            C_out  : out STD_LOGIC
        );
    end component;

    signal C1,C2,C3 : std_logic;
```

```

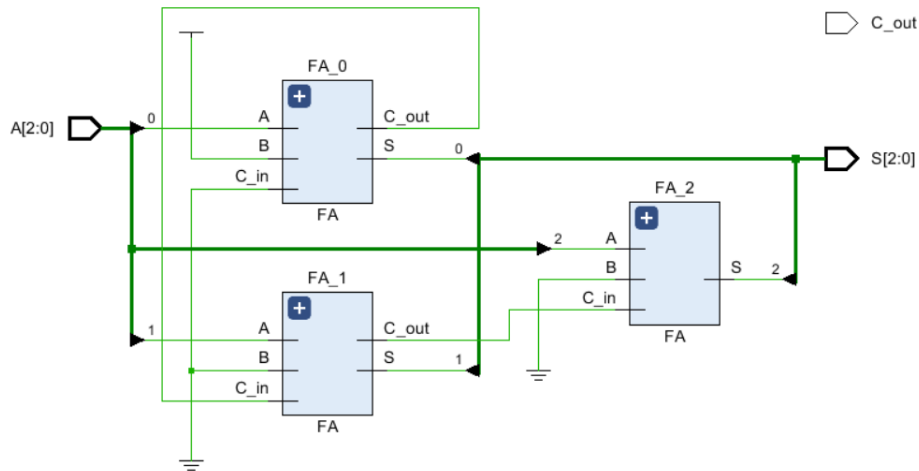
begin
    FA_0 : FA
        port map(
            A => A(0),
            B => '1',
            C_in => '0',
            S => S(0),
            C_out => C1
        );

    FA_1 : FA
        port map(
            A => A(1),
            B => '0',
            C_in => C1,
            S => S(1),
            C_out => C2
        );

    FA_2 : FA
        port map(
            A => A(2),
            B => '0',
            C_in => C2,
            S => S(2),
            C_out => C3
        );
end Behavioral;

```


RTL Schematic diagram of Adder 3bit



Behavioral simulation Code for Adder 3bit

```
-- Company:
-- Engineer:
--
-- Create Date: 04/30/2026 10:55:36 AM
-- Design Name:
-- Module Name: TB_3bitAdder - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
```

```
library IEEE;
```

```

use IEEE.STD_LOGIC_1164.ALL;

entity TB_3bitAdder is
-- Testbench entities are always empty
end TB_3bitAdder;

architecture Behavioral of tb_3bitAdder is

    -- Component Declaration for the Unit Under Test (UUT)
    component Adder_3bit
        Port ( A : in STD_LOGIC_VECTOR (2 downto 0);
              S : out STD_LOGIC_VECTOR (2 downto 0);
              C_out : out STD_LOGIC);
    end component;

    -- Inputs
    signal A : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');

    -- Outputs
    signal S : STD_LOGIC_VECTOR(2 downto 0);
    signal C_out : STD_LOGIC;

begin

    -- Instantiate the Unit Under Test (UUT)
    UUT: Adder_3bit port map (
        A => A,
        S => S,
        C_out => C_out
    );

    -- Stimulus process
    stim_proc: process
    begin
        -- Test case 1: 0 + 1 = 1
        A <= "000";
        wait for 20 ns;

        -- Test case 2: 3 + 1 = 4
        A <= "011";
        wait for 20 ns;

        -- Test case 3: 6 + 1 = 7
        A <= "110";
        wait for 20 ns;
    end process;
end architecture;

```

```

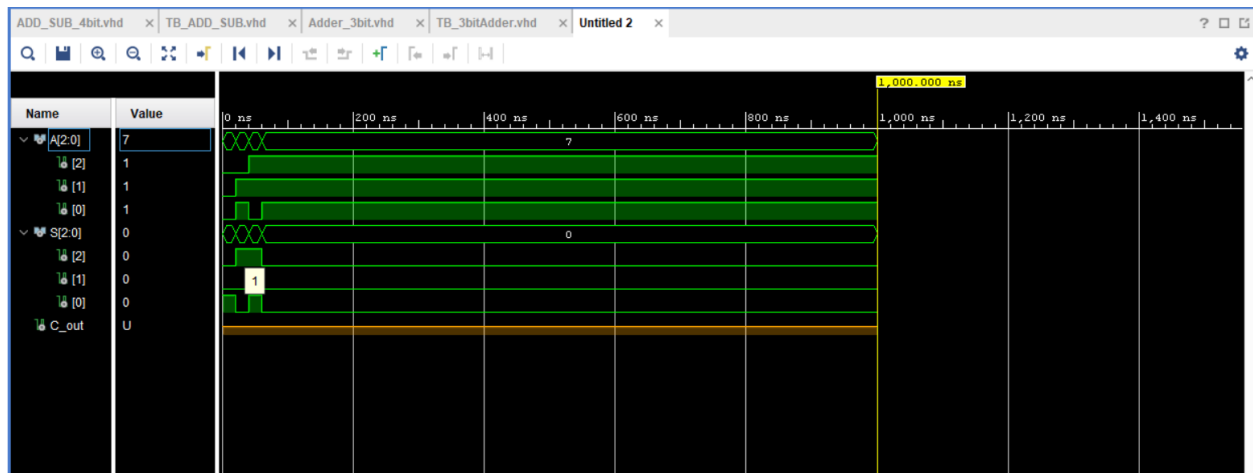
-- Test case 4: 7 + 1 = 0 (Expected C_out = '1')
A <= "111";
wait for 20 ns;

wait; -- Suspend simulation
end process;

end Behavioral;

```

Timing Diagram for Adder 3bit



18. Lookup Table

Design source VHDL code of LUT table

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity LUT_16_7 is
    Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
          data : out STD_LOGIC_VECTOR (6 downto 0));
end LUT_16_7;

architecture Behavioral of LUT_16_7 is

    type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
    signal sevenSegment_ROM : rom_type := (
        "1000000", -- 0
        "1111001", -- 1
        "0100100", -- 2
        "0110000", -- 3

```

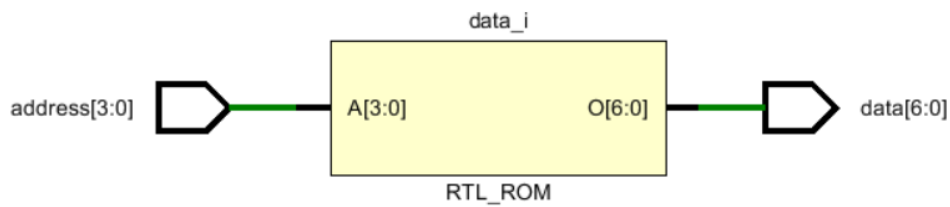
```

    "0011001", -- 4
    "0010010", -- 5
    "0000010", -- 6
    "1111000", -- 7
    "0000000", -- 8
    "0010000", -- 9
    "0001000", -- a
    "0000011", -- b
    "1000110", -- c
    "0100001", -- d
    "0000110", -- e
    "0001110" - f
);
begin
data <= sevenSegment_ROM(to_integer(unsigned(address)));

end Behavioral;

```

RTL Schematic diagram of LUT Table



Constraint File for Version 1

```

## Clock
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports clk]

set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

## reg[3:0] â†’ LEDs
set_property PACKAGE_PIN U16 [get_ports {reg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg[0]}]
set_property PACKAGE_PIN E19 [get_ports {reg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg[1]}]
set_property PACKAGE_PIN U19 [get_ports {reg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg[2]}]
set_property PACKAGE_PIN V19 [get_ports {reg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {reg[3]}]

## overflow, zero, carry â†’ status indicators
set_property PACKAGE_PIN N3 [get_ports {overflow}]
set_property IOSTANDARD LVCMOS33 [get_ports {overflow}]
set_property PACKAGE_PIN P1 [get_ports {zero}]
set_property IOSTANDARD LVCMOS33 [get_ports {zero}]
set_property PACKAGE_PIN L1 [get_ports {carry}]
set_property IOSTANDARD LVCMOS33 [get_ports {carry}]

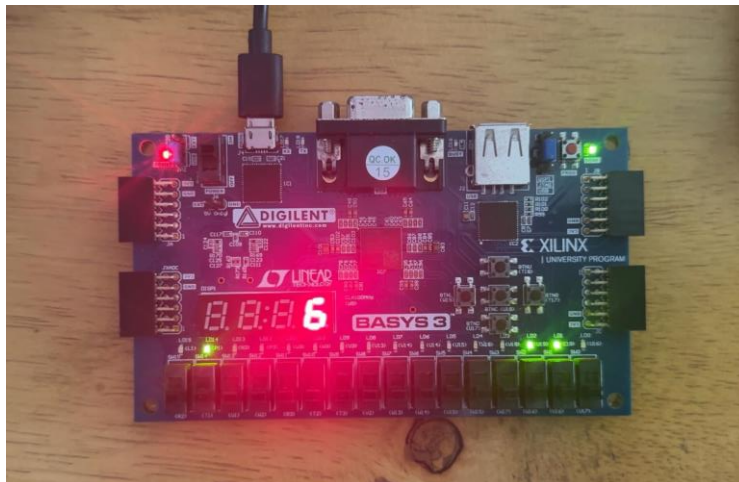
## 7-Segment Display segments: display[6:0]
set_property PACKAGE_PIN W7 [get_ports {display[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[0]}]
set_property PACKAGE_PIN W6 [get_ports {display[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[1]}]
set_property PACKAGE_PIN U8 [get_ports {display[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[2]}]
set_property PACKAGE_PIN V8 [get_ports {display[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[3]}]
set_property PACKAGE_PIN U5 [get_ports {display[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[4]}]
set_property PACKAGE_PIN V5 [get_ports {display[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[5]}]
set_property PACKAGE_PIN U7 [get_ports {display[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[6]}]
# set_property PACKAGE_PIN V7 [get_ports dp]

```

```
# set_property IOSTANDARD LVCMOS33 [get_ports dp]
## Anodes: anode[3:0]
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]
```

Tasks Completed and Results

Added 1, 2, 3 and consecutively showed results



Nano processor version 02 with extended qualifications

Components and instructions

1. Allocated reset button
 - a. The nano processor can be reset by pressing the btnC.
 - b. The clock speed of the internal clock of the machine was reduced from 100MHz to 0.5MHz using a slow clock to make the calculation process visible to the naked eye.
 - c. In order to reset the nano processor, it is necessary to press and hold the reset button for at least 2 seconds.
2. LED signal
 - a. LED0-LED3 Output of R7 register in Register Bank
 - i. LED0-LED3 → Outputs a 4-bit number as a signed number in two's complement method
 - b. LED6-LED7 Output of the Comparator
 - i. LED9 → Equal
 - ii. LED10 → Greater
 - iii. LED11 → Lesser
 - c. LED12-LED15 Flags of 4-bit Add/Subtract Unit
 - i. LED12 → Sign Flag
 - ii. LED13 → Overflow Flag
 - iii. LED14 → Zero Flag
 - iv. LED15 → Carry Flag
3. 7 Segment Display
 - a. The rightmost segment of the 7-Segment display is used to display the magnitude of the output from the R7 register in the Register Bank.
4. Expected behavior of the program
 - a. After the calculation is performed the process will be held because of the jump instruction implemented.
 - b. Therefore, it is necessary to manually reset the nano processor to perform the instructions hardcoded in the ROM.

A Comparative Study of Version 1 and Version 2

The transition from Nano Processor Version 1 to Version 2 showcases a substantial enhancement in both functionality and architectural design. Version 1 establishes a foundation with a minimal instruction set and a simpler architecture, while Version 2 expands the processor's capabilities with a wider instruction set, improved data handling, and enhanced logical processing, effectively bridging the gap between academic exercises and practical embedded processor design.

In this updated design, 1-bit comparators were created to compare two 1-bit numbers. By utilizing four of these 1-bit comparators, we developed a 4-bit comparator that functions as a sign-magnitude comparator. Notably, apart from the comparator, the inputs of other arithmetic units are interconnected, allowing for simultaneous calculations.

To facilitate this, an 8-way 4-bit multiplexer and a 3-bit output enabler are employed to produce outputs from all operation units concurrently. The design includes the capability to multiply a 4-bit number by a 7-bit output. However, handling this output presents challenges. To address this limitation, Version 2 incorporates an operation that specifically multiplies only the last two digits of each input.

	Version 01	Version 02
Program Counter	3-bit (addresses up to 8 memory locations)	Expanded to 4 bit (16 memory addresses)
ROM	8 locations	16 locations
Instruction Size	12 bits	14 bits
Memory Selector Size	3 bits for adder and Memory Selector	4 bits for adder and Memory Selector
Supported Instructions	• Add • Mov • Neg • JZR	• Add • Mov • Neg • JZR • Multiply partially • Bitwise OR • Bitwise AND • Bitwise XOR • Bitwise NOT • Right Shift by 1 or 2 positions.

Optimized Slice Logic and Design Primitives

- Slice Logic

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	80	0	20800	0.38
LUT as Logic	80	0	20800	0.38
LUT as Memory	0	0	9600	0.00
Slice Registers	50	0	41600	0.12
Register as Flip Flop	50	0	41600	0.12
Register as Latch	0	0	41600	0.00
F7 Muxes	0	0	16300	0.00
F8 Muxes	0	0	8150	0.00

- Design Primitives

7. Primitives

Ref Name	Used	Functional Category
FDRE	38	Flop & Latch
LUT4	35	LUT
LUT2	29	LUT
OBUF	22	IO
LUT3	17	LUT
LUT6	14	LUT
FDCE	12	Flop & Latch
CARRY4	8	Carrylogic
LUT5	5	LUT
LUT1	2	LUT
IBUF	2	IO
BUFG	1	Clock

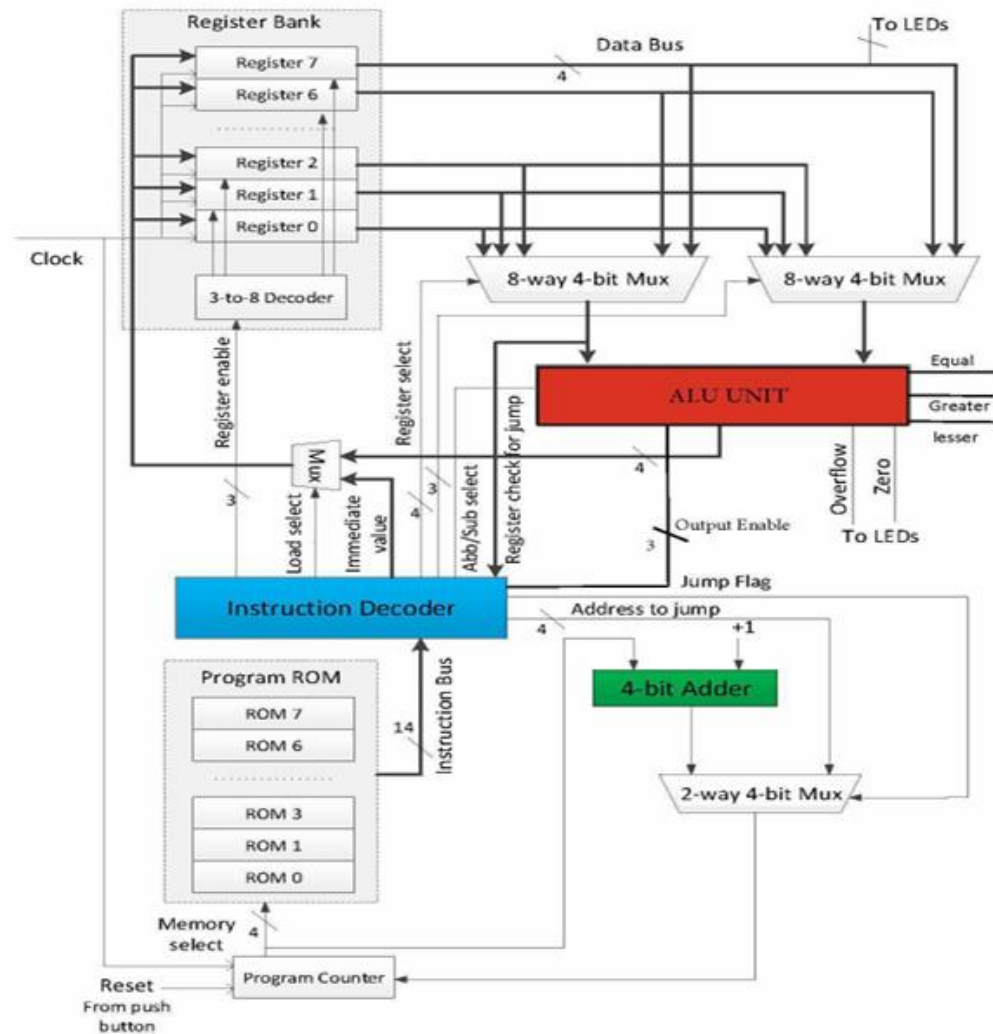
Executable Instruction Set

We extended the instruction set by the following ways. Although in the previous design, instruction set has only 12 bits, In the next version, we extend the instruction code by 2 bits and first 4bit of instruction line use to implement more instructions.

Instruction	Description	Format (14-bit Instruction)
ADD Ra, Rb	Add values in registers Ra and Rb and store the result in Ra, i.e. $R_a \leftarrow R_a + R_b$, $R_a, R_b \in [0, 7]$	0000 $R_a R_a R_a R_b R_b R_b$ 0000
NEG R	2's Complement of registers R. i.e. $R \leftarrow -R$, $R \in [0, 7]$	0001 RRR 000 0000
MOVI R, d	Move Immediate Value d, to register R, i.e. $R \leftarrow d$, $R \in [0, 7]$ & $d \in [0, 15]$	0010 RRR 000 dddd
JZR R, d	Jump if value in register R is 0, $R, d \in [0, 7]$ i.e. If $R == 0$; $PC \leftarrow d$; Else; $PC \leftarrow PC + 1$	0011 RRR 000 dddd
COM Ra, R b	Compare values in registers Ra & Rb: $R_a, R_b \in [0, 7]$	0100 $R_a R_a R_a R_b R_b R_b$ 0000
MUL Ra, Rb	Multiply last two digits of values in registers Ra & Rb and store the result in Ra, i.e. $R_a \leftarrow R_a[1:0] * R_b[1:0]$; $R_a, R_b \in [0, 7]$	0101 $R_a R_a R_a R_b R_b R_b$ 0000
BWA Ra, Rb	Get Bitwise AND of values in registers R_a & R_b and store the result in R_a ; $R_a \leftarrow R_a \& R_b$; $R_a, R_b \in [0, 7]$	0110 $R_a R_a R_a R_b R_b R_b$ 0000
BWO Ra, Rb	Get Bitwise OR of values in registers R_a & R_b and store the result in R_a ; $R_a \leftarrow R_a R_b$; $R_a, R_b \in [0, 7]$	0111 $R_a R_a R_a R_b R_b R_b$ 0000

BWX Ra, Rb	Get Bitwise XOR of values in registers R_a & R_b and store the result in R_a ; $R_a \leftarrow R_a \wedge R_b$; $R_a, R_b \in [0, 7]$	1000 $R_a R_a R_a R_b R_b R_b$ 0000
NOT R	Get Bitwise NOT of value in Register R and Store the result in R i.e. $R \leftarrow \sim R$, $R \in [0, 7]$	1001 RRR 000 0000
RS1 R	Right Shift by one place the value in Register R and store the result in R; $R \leftarrow R >> 1$, $R \in [0, 7]$	1010 RRR 000 0000
RS2 R	Right Shift by two places the value in Register R and store the result in R; $R \leftarrow R >> 2$, $R \in [0, 7]$	1011 RRR 000 0000

- Design Components



28. nano processor version_2

Design source VHDL code of nano processor version 2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity NanoProcessor is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          reg : out STD_LOGIC_VECTOR (3 downto 0);
          zero : out STD_LOGIC;
```

```

        overflow : out STD_LOGIC;
        display : out STD_LOGIC_VECTOR (6 downto 0);
        anode : out STD_LOGIC_VECTOR (3 downto 0);
        carry : out STD_LOGIC;
        var0, var1 : out STD_LOGIC_VECTOR (3 downto 0)
    );
end NanoProcessor;

```

architecture Behavioral of NanoProcessor is

```

component LUT_16_7
    Port( address : in STD_LOGIC_VECTOR(3 downto 0);
          data : out STD_LOGIC_VECTOR(6 downto 0));
end component;

```

```

component Register_Bank
    Port( Clk: in STD_LOGIC;
          Reset: in STD_LOGIC;
          D : in STD_LOGIC_VECTOR(3 downto 0);
          I : in STD_LOGIC_VECTOR(2 downto 0);
          R0 : out STD_LOGIC_VECTOR(3 downto 0);
          R1 : out STD_LOGIC_VECTOR(3 downto 0);
          R2 : out STD_LOGIC_VECTOR(3 downto 0);
          R3 : out STD_LOGIC_VECTOR(3 downto 0);
          R4 : out STD_LOGIC_VECTOR(3 downto 0);
          R5 : out STD_LOGIC_VECTOR(3 downto 0);
          R6 : out STD_LOGIC_VECTOR(3 downto 0);
          R7 : out STD_LOGIC_VECTOR(3 downto 0));
end component;

```

```

component ADD_SUB_4bit
    Port(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          Enable : in STD_LOGIC;
          Operation : in STD_LOGIC;
          C_out : out STD_LOGIC;
          S: out STD_LOGIC_VECTOR(3 downto 0);
          Overflow : out STD_LOGIC;
          Zero : out STD_LOGIC);
end component;

```

```

Component Comparator_4bit is
    Port ( A, B : in STD_LOGIC_VECTOR(3 downto 0);
          Ena : in STD_LOGIC;
          Equal, Greater, Less : out STD_LOGIC );

```

```
end Component;
```

```
Component Multiplier_2bit is
```

```
    Port ( A4 : in STD_LOGIC_VECTOR (3 downto 0);  
          B4 : in STD_LOGIC_VECTOR (3 downto 0);  
          Enable : in STD_LOGIC;  
          Y : out STD_LOGIC_VECTOR (3 downto 0));
```

```
end Component;
```

```
component Instruction_Decoder
```

```
    Port( I : in STD_LOGIC_VECTOR(12 downto 0);  
          Reg_Check_Jump: in STD_LOGIC_VECTOR(3 downto 0);  
          Load_Select : out STD_LOGIC;  
          Operation_En : out STD_LOGIC_VECTOR(3 downto 0);  
          Imm_Value : out STD_LOGIC_VECTOR(3 downto 0);  
          Reg_Enable : out STD_LOGIC_VECTOR(2 downto 0);  
          Reg_Select_1 : out STD_LOGIC_VECTOR(2 downto 0);  
          Reg_Select_2 : out STD_LOGIC_VECTOR(2 downto 0);  
          Add_Sub : out STD_LOGIC;  
          Jump_Flag : out STD_LOGIC;  
          Address : out STD_LOGIC_VECTOR(2 downto 0)  
    );
```

```
end component;
```

```
component Slow_Clock
```

```
    Port(Clk_in : in STD_LOGIC;  
          Clk_out: out STD_LOGIC);
```

```
end component;
```

```
component Program_ROM
```

```
    Port( address : in STD_LOGIC_VECTOR(2 downto 0);  
          instruction_code : out STD_LOGIC_VECTOR(12 downto 0));
```

```
end component;
```

```
component MUX_2_way_3_bit
```

```
    Port( Selector: in STD_LOGIC;  
          A : in STD_LOGIC_VECTOR(2 downto 0);  
          B : in STD_LOGIC_VECTOR(2 downto 0);  
          Y : out STD_LOGIC_VECTOR(2 downto 0));
```

```
end component;
```

```
component MUX_2_way_4_bit
```

```
    Port( Selector: in STD_LOGIC;  
          A : in STD_LOGIC_VECTOR(3 downto 0);  
          B : in STD_LOGIC_VECTOR(3 downto 0);
```

```

        Y : out STD_LOGIC_VECTOR(3 downto 0));
end component;

```

```

component MUX_8_way_4_bit
  Port( Selector : in STD_LOGIC_VECTOR(2 downto 0);
        A0 : in STD_LOGIC_VECTOR(3 downto 0);
        A1 : in STD_LOGIC_VECTOR(3 downto 0);
        A2 : in STD_LOGIC_VECTOR(3 downto 0);
        A3 : in STD_LOGIC_VECTOR(3 downto 0);
        A4 : in STD_LOGIC_VECTOR(3 downto 0);
        A5 : in STD_LOGIC_VECTOR(3 downto 0);
        A6 : in STD_LOGIC_VECTOR(3 downto 0);
        A7 : in STD_LOGIC_VECTOR(3 downto 0);
        Y : out STD_LOGIC_VECTOR(3 downto 0));
end component;

```

```

component Adder_3bit
  Port( A : in STD_LOGIC_VECTOR(2 downto 0);
        S : out STD_LOGIC_VECTOR(2 downto 0);
        C_out : out STD_LOGIC);
end component;

```

```

component PC_3_bit
  Port( D : in STD_LOGIC_VECTOR(2 downto 0);
        Res : in STD_LOGIC;
        Clk : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR(2 downto 0));
end component;

```

```

signal Load_Select, Add_Sub_Selector, Jump_Flag, Overflow_0, Z_out:
STD_LOGIC;
signal Ins_Bus : STD_LOGIC_VECTOR(12 downto 0);
signal O,b,Add_out,Reg_Enable,Reg_Select_1,Reg_Select_2,OpCode:
STD_LOGIC_VECTOR(2 downto 0);
signal Address : STD_LOGIC_VECTOR(2 downto 0);
signal Imm_Value, R0,R1,R2,R3,R4,R5,R6,R7,S,X,Y1,Y2,MultiplierOutput:
STD_LOGIC_VECTOR(3 downto 0);
signal Clk_out : STD_LOGIC;
signal Adder_En, Comparator_En,Multiplier_En : STD_LOGIC;
signal Operation_En,seg_Address : STD_LOGIC_VECTOR(3 DOWNT0 0);
SIGNAL Equal, Greater, Less : STD_LOGIC;

```

```

begin

```

```

SlowClock : Slow_Clock
    Port map (
        Clk_in => clk,
        Clk_out => Clk_out);

LUT : LUT_16_7
    Port map(
        address => seg_Address,
        data => display);

ProgramCounter : PC_3_bit
    Port map(
        D => b,
        Q => O,
        Clk => Clk_out,
        Res => reset);

ProgramRom : Program_ROM
    Port map(
        address => O,
        instruction_code => Ins_Bus);

InstructionDecoder : Instruction_Decoder
    Port map(
        I => Ins_Bus,
        Reg_Check_Jump => Y1,
        Load_Select => Load_Select,
        Operation_En => Operation_En,
        Imm_Value => Imm_Value,
        Reg_Enable => Reg_Enable,
        Reg_Select_1 => Reg_Select_1,
        Reg_Select_2 => Reg_Select_2,
        Add_Sub => Add_Sub_Selector,
        Jump_Flag => Jump_Flag,
        Address => Address
    );

MUX_2_to_1_4bit_1 : MUX_2_way_4_bit
    Port map(
        Selector => Load_Select,
        A => S,
        B => Imm_Value,
        Y => X);

RegisterBank : Register_Bank

```



```

Port map(
  Clk => Clk_out,
  Reset => reset,
  D => X,
  R0 => R0,
  R1 => R1,
  R2 => R2,
  R3 => R3,
  R4 => R4,
  R5 => R5,
  R6 => R6,
  R7 => R7,
  I => Reg_Enable);

```

MUX_8_to_1_4bit_0 : MUX_8_way_4_bit

```

Port map(
  Selector => Reg_Select_1,
  A0 => R0,
  A1 => R1,
  A2 => R2,
  A3 => R3,
  A4 => R4,
  A5 => R5,
  A6 => R6,
  A7 => R7,
  Y => Y1);

```

MUX_8_to_1_4bit_1 : MUX_8_way_4_bit

```

Port map(
  Selector => Reg_Select_2,
  A0 => R0,
  A1 => R1,
  A2 => R2,
  A3 => R3,
  A4 => R4,
  A5 => R5,
  A6 => R6,
  A7 => R7,
  Y => Y2);

```

AddSub : ADD_SUB_4bit

```

Port map(
  A => Y1,
  B => Y2,
  Enable => Adder_En,

```

```

        Operation => Add_Sub_Selector,
        C_out => carry,
        S => S,
        Overflow => Overflow_0,
        Zero => Z_out);

Comparator : Comparator_4bit
    Port map( A => Y1,
              B => Y2,
              Ena => Comparator_En,
              Equal => Equal,
              Greater => Greater,
              Less => Less);

Multiplier : Multiplier_2bit
    Port map ( A4 => Y1,
              B4 => Y2,
              Enable => Multiplier_En,
              Y => MultiplierOutput );

Mux_2_to_1_3bit_0 : MUX_2_way_3_bit
    Port map(
        Selector => Jump_Flag,
        A => Add_Out,
        B => Address,
        Y => b);

Adder3bit : Adder_3bit
    Port map(
        A => O,
        S => Add_Out);

process(OpCode, Less, Greater, Equal, MultiplierOutput)
begin

    OpCode <= Ins_Bus(12 downto 10);

    seg_Address <= "0000";
    if OpCode = "100" then
        if Less = '1' then
            anode <= "1101";
            seg_Address <= "0001";
        elsif Greater = '1' then
            anode <= "0111";
            seg_Address <= "0001";
        end if;
    end if;
end process;

```

```

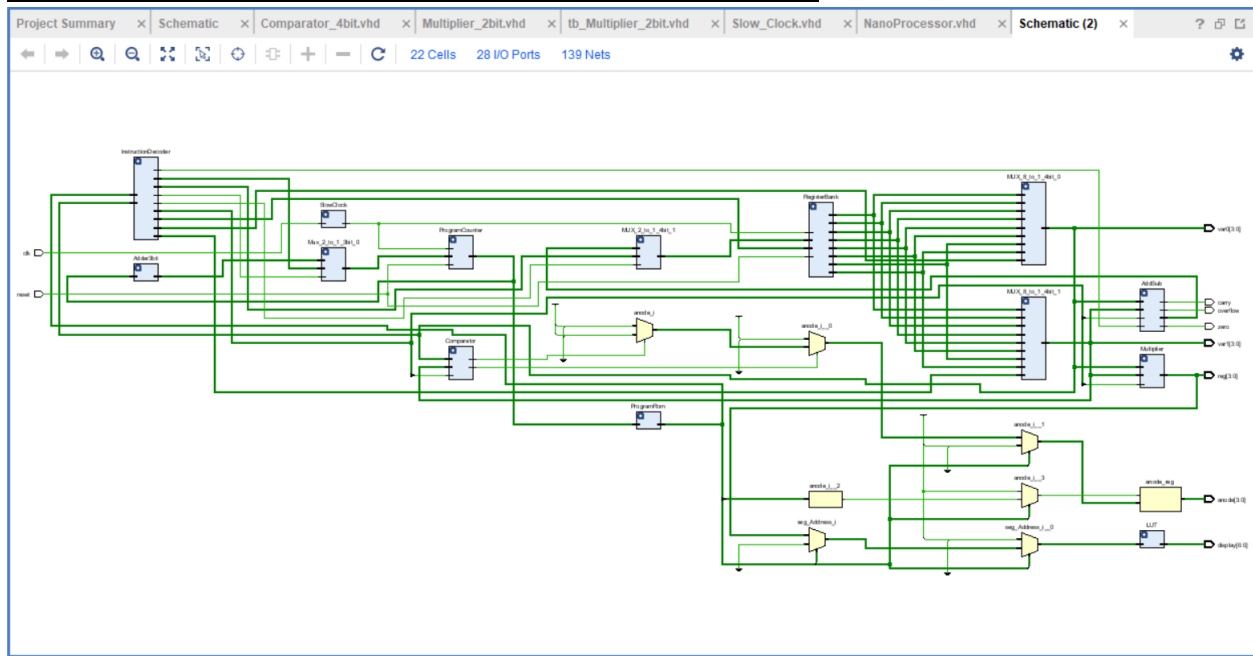
        else
            anode <= "1011";
            seg_Address <= "0001";
        end if;
    elsif OpCode = "111" then
        anode <= "1110";
        seg_Address <= MultiplierOutput;
    end if;
end process;

Adder_En <= Operation_En(0);
Comparator_En <= Operation_En(1);
Multiplier_En <= Operation_En(2);
reg <= MultiplierOutput;
zero <= Z_out;
overflow <= Overflow_0;

var0 <= Y1;
var1 <= Y2;

```

RTL Schematic diagram of nano processor version 2



Behavioral simulation Code for nano processor version 2

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Nanoprocessor is

```

```
-- Port ( );  
end TB_Nanoprocessor;
```

architecture Behavioral of TB_Nanoprocessor is

```
component NanoProcessor
```

```
    Port (clk : in STD_LOGIC;  
          reset : in STD_LOGIC;  
          reg : out STD_LOGIC_VECTOR (3 downto 0);  
          display : out STD_LOGIC_VECTOR (6 downto 0);  
          zero : out STD_LOGIC;  
          overflow : out STD_LOGIC;  
          anode : out STD_LOGIC_VECTOR (3 downto 0);  
          carry: out STD_LOGIC    );
```

```
end component;
```

```
Signal clk,reset,zero,overflow,carry : STD_LOGIC;
```

```
signal reg,anode : STD_LOGIC_VECTOR (3 downto 0);
```

```
signal display : STD_LOGIC_VECTOR (6 downto 0);
```

```
begin
```

```
UUT : NanoProcessor
```

```
    Port Map (  
        clk => clk,  
        reset => reset,  
        reg => reg,  
        zero => zero,  
        overflow => overflow,  
        carry => carry,  
        display => display,  
        anode => anode  
    );
```

```
process
```

```
begin
```

```
    clk <= '0';
```

```
    wait for 50 ns;
```

```
    clk <= '1';
```

```
    wait for 50 ns;
```

```
end process;
```

```
process
```

```
begin
```

```
    reset <= '1';
```

```
    wait for 100ns;
```

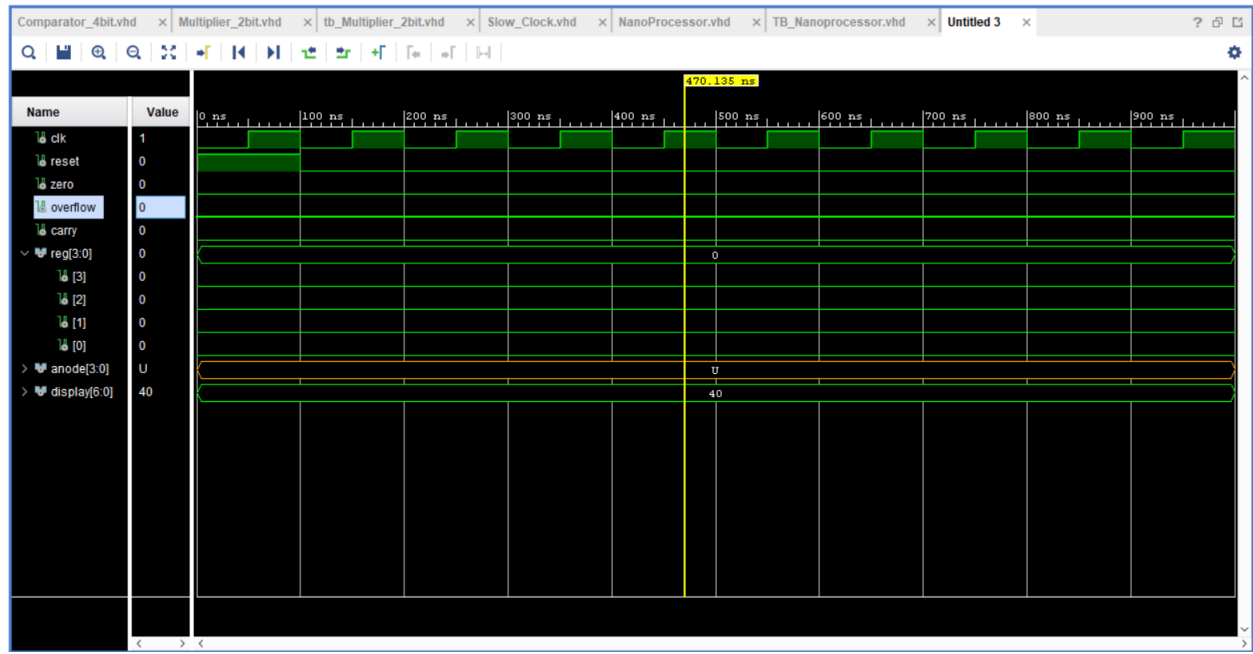
```
    reset <= '0';
```

```

        wait;
    end process;
end Behavioral;

```

Timing Diagram for nano processor version 2



2. comparator_1bit

Design source VHDL code of comparator 1bit

```

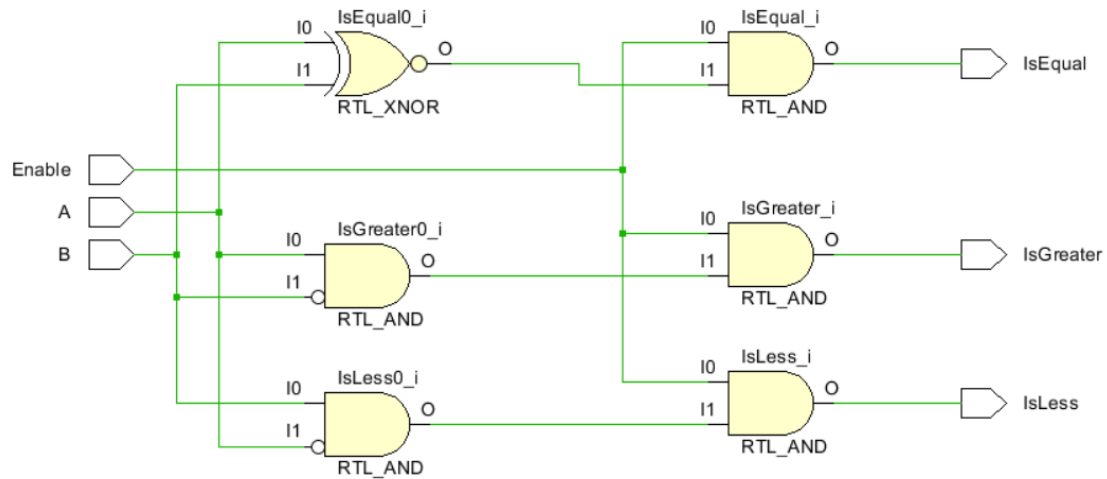
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Comparator_1bit is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          Enable : in STD_LOGIC;
          IsEqual : out STD_LOGIC;
          IsGreater : out STD_LOGIC;
          IsLess : out STD_LOGIC );
end Comparator_1bit;

architecture Behavioral of Comparator_1bit is
begin
    IsEqual    <= Enable AND (A XNOR B);
    IsGreater  <= Enable AND (A AND NOT B);
    IsLess     <= Enable AND (B AND NOT A);
end Behavioral;

```

RTL Schematic diagram of comparator 1bit



Behavioral simulation Code for comparator 1bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Comparator_1bit_sim is
end Comparator_1bit_tb;

architecture Behavioral of Comparator_1bit_sim is

    component Comparator_1bit
        Port (
            A : in STD_LOGIC;
            B : in STD_LOGIC;
            Enable : in STD_LOGIC;
            IsEqual : out STD_LOGIC;
            IsGreater : out STD_LOGIC;
            IsLess : out STD_LOGIC
        );
    end component;

end architecture;
```

```

signal A, B, Enable : std_logic := '0';
signal IsEqual, IsGreater, IsLess : std_logic;

begin

    DUT: Comparator_1bit
        port map (
            A => A,
            B => B,
            Enable => Enable,
            IsEqual => IsEqual,
            IsGreater => IsGreater,
            IsLess => IsLess
        );

    process
    begin
        -- Enable OFF: All outputs should be 0
        Enable <= '0';
        A <= '0'; B <= '0'; wait for 100 ns;
        A <= '0'; B <= '1'; wait for 100 ns;
        A <= '1'; B <= '0'; wait for 100 ns;
        A <= '1'; B <= '1'; wait for 100 ns;

        -- Enable ON: Check all combinations
        Enable <= '1';
        A <= '0'; B <= '0'; wait for 100 ns; -- Equal
        A <= '0'; B <= '1'; wait for 100 ns; -- Less
        A <= '1'; B <= '0'; wait for 100 ns; -- Greater
        A <= '1'; B <= '1'; wait for 100 ns; -- Equal

        -- Hold simulation
        wait;
    end process;

end Behavioral;

```

Timing Diagram for comparator 1bit



3. comparator_4bit

Design source VHDL code of comparator 4bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Comparator_4bit is
    Port ( A, B : in STD_LOGIC_VECTOR(3 downto 0);
          Ena : in STD_LOGIC;
          Equal, Greater, Less : out STD_LOGIC );
end Comparator_4bit;

architecture Structural of Comparator_4bit is

    component Comparator_1bit
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              Enable : in STD_LOGIC;
              IsEqual : out STD_LOGIC;
              IsGreater : out STD_LOGIC;
              IsLess : out STD_LOGIC );
    end component;

    signal eq : STD_LOGIC_VECTOR(3 downto 0);
    signal gt : STD_LOGIC_VECTOR(3 downto 0);
    signal lt : STD_LOGIC_VECTOR(3 downto 0);
    signal en : STD_LOGIC_VECTOR(3 downto 0);

begin

    en(3) <= '1';

```



```

en(2) <= eq(3);
en(1) <= eq(3) and eq(2);
en(0) <= eq(3) and eq(2) and eq(1);

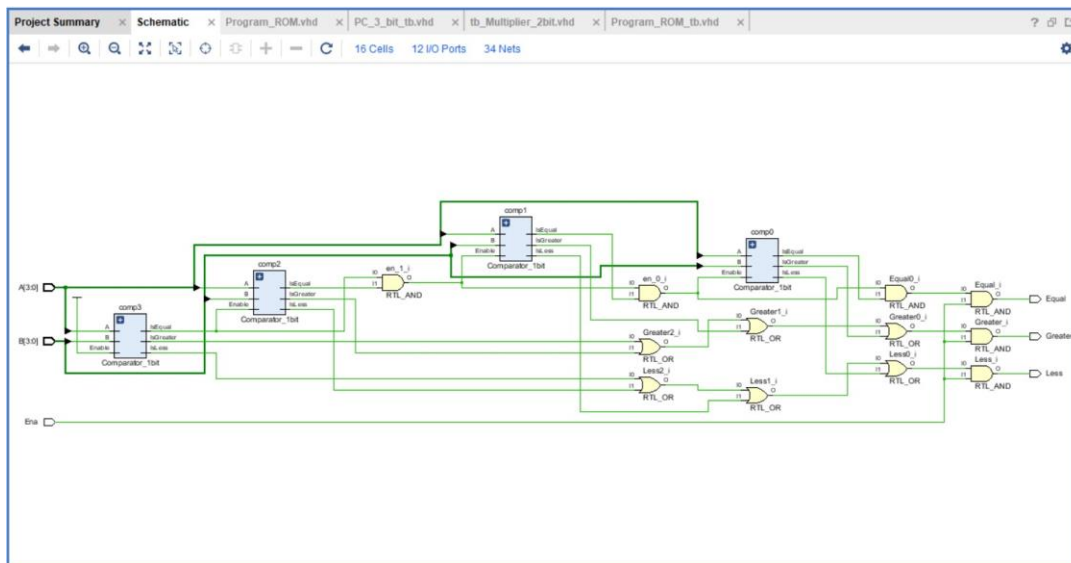
-- Instantiate 1-bit comparators
comp3: Comparator_1bit port map(A => A(3), B => B(3), Enable =>
en(3), IsEqual => eq(3), IsGreater => gt(3), IsLess => lt(3));
comp2: Comparator_1bit port map(A => A(2), B => B(2), Enable =>
en(2), IsEqual => eq(2), IsGreater => gt(2), IsLess => lt(2));
comp1: Comparator_1bit port map(A => A(1), B => B(1), Enable =>
en(1), IsEqual => eq(1), IsGreater => gt(1), IsLess => lt(1));
comp0: Comparator_1bit port map(A => A(0), B => B(0), Enable =>
en(0), IsEqual => eq(0), IsGreater => gt(0), IsLess => lt(0));

-- Final outputs: prioritize MSB to LSB
Greater <= ( gt(3) or gt(2) or gt(1) or gt(0) ) AND Ena;
Less    <= ( lt(3) or lt(2) or lt(1) or lt(0) ) AND Ena;
Equal   <= ( eq(3) and eq(2) and eq(1) and eq(0) ) AND Ena;

end Structural;

```

RTL Schematic diagram of comparator 4bit



Behavioral simulation Code for comparator 4bit

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity Comparator_4bit is
    Port ( A, B : in STD_LOGIC_VECTOR(3 downto 0);
          Ena : in STD_LOGIC;
          Equal, Greater, Less : out STD_LOGIC );
end Comparator_4bit;

```

architecture Structural of Comparator_4bit is

```

    component Comparator_1bit
        Port ( A : in STD_LOGIC;
              B : in STD_LOGIC;
              Enable : in STD_LOGIC;
              IsEqual : out STD_LOGIC;
              IsGreater : out STD_LOGIC;
              IsLess : out STD_LOGIC );
    end component;

```

```

    signal eq : STD_LOGIC_VECTOR(3 downto 0);
    signal gt : STD_LOGIC_VECTOR(3 downto 0);
    signal lt : STD_LOGIC_VECTOR(3 downto 0);
    signal en : STD_LOGIC_VECTOR(3 downto 0);

```

begin

```

    en(3) <= '1';
    en(2) <= eq(3);
    en(1) <= eq(3) and eq(2);
    en(0) <= eq(3) and eq(2) and eq(1);

```

```

    -- Instantiate 1-bit comparators
    comp3: Comparator_1bit port map(A => A(3), B => B(3), Enable =>
en(3), IsEqual => eq(3), IsGreater => gt(3), IsLess => lt(3));
    comp2: Comparator_1bit port map(A => A(2), B => B(2), Enable =>
en(2), IsEqual => eq(2), IsGreater => gt(2), IsLess => lt(2));
    comp1: Comparator_1bit port map(A => A(1), B => B(1), Enable =>
en(1), IsEqual => eq(1), IsGreater => gt(1), IsLess => lt(1));
    comp0: Comparator_1bit port map(A => A(0), B => B(0), Enable =>
en(0), IsEqual => eq(0), IsGreater => gt(0), IsLess => lt(0));

```

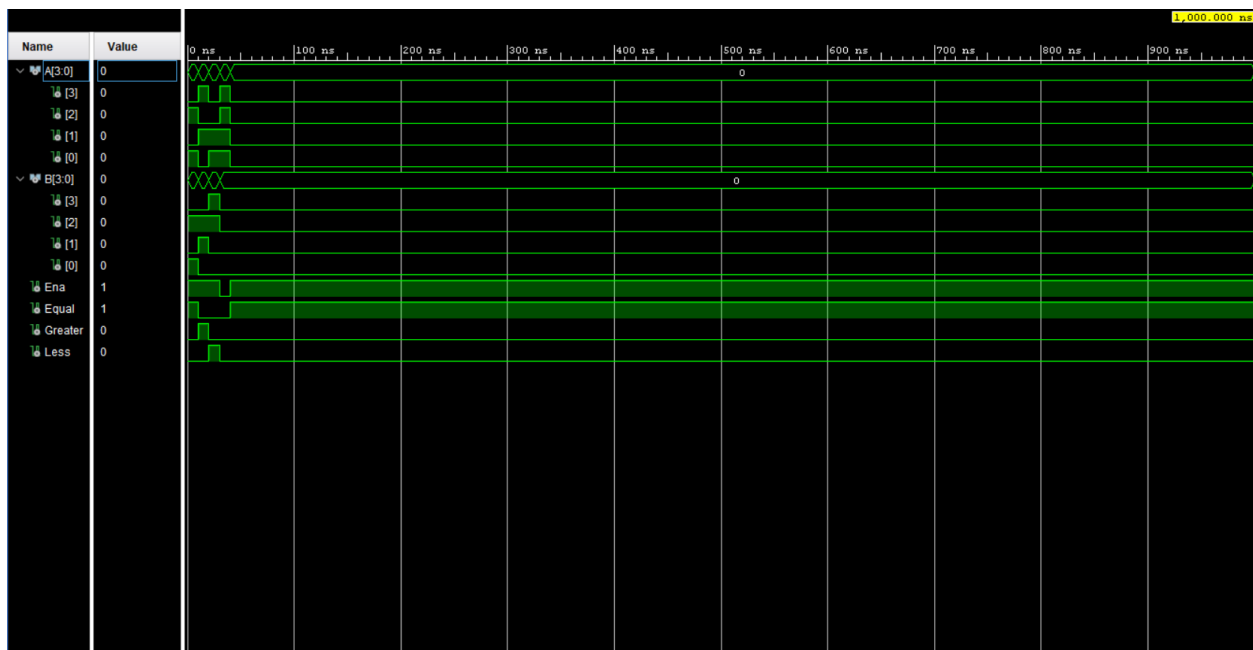
```

    -- Final outputs: prioritize MSB to LSB
    Greater <= ( gt(3) or gt(2) or gt(1) or gt(0) ) AND Ena;
    Less    <= ( lt(3) or lt(2) or lt(1) or lt(0) ) AND Ena;
    Equal   <= ( eq(3) and eq(2) and eq(1) and eq(0) ) AND Ena;

```

end Structural;

Timing Diagram for comparator 4bit



4. Multiplier_2bit

Design source VHDL code of Multiplier 2bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Multiplier_2bit is
    Port ( A4 : in STD_LOGIC_VECTOR (3 downto 0);
          B4 : in STD_LOGIC_VECTOR (3 downto 0);
          Enable : in STD_LOGIC;
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end Multiplier_2bit;

architecture Behavioral of Multiplier_2bit is
    signal pp0, pp1, pp2, pp3 : std_logic;
    signal sum1, carry1, sum2, carry2 : std_logic;
    signal A, B : STD_LOGIC_VECTOR (1 DOWNTO 0);
    signal tempY : std_logic_vector(3 downto 0);

begin
    -- Only use 2 LSBs
    A <= A4(1 DOWNTO 0);
    B <= B4(1 DOWNTO 0);
```

```

-- Partial products
pp0 <= A(0) and B(0);
pp1 <= A(1) and B(0);
pp2 <= A(0) and B(1);
pp3 <= A(1) and B(1);

sum1  <= pp1 xor pp2;
carry1 <= pp1 and pp2;

sum2  <= pp3 xor carry1;
carry2 <= pp3 and carry1;

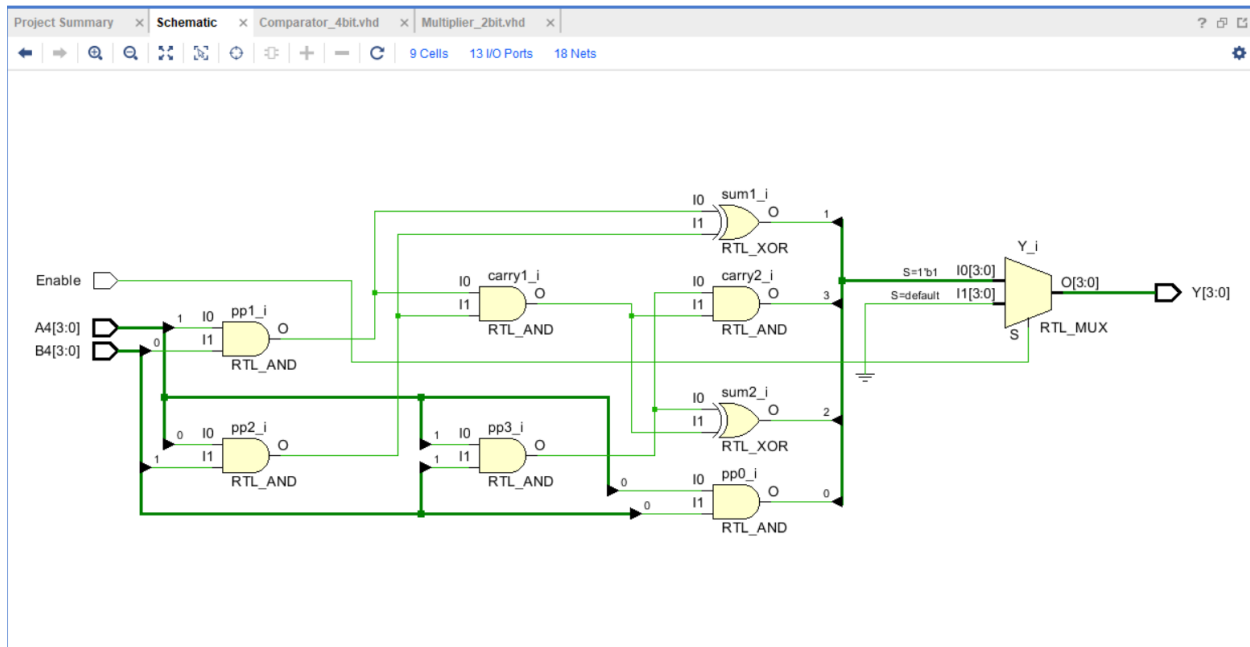
tempY(0) <= pp0;
tempY(1) <= sum1;
tempY(2) <= sum2;
tempY(3) <= carry2;

process(tempY, Enable)
begin
    if Enable = '1' then
        Y <= tempY;
    else
        Y <= (others => '0');
    end if;
end process;

end Behavioral;

```

RTL Schematic diagram of Multiplier 2bit



Behavioral simulation Code for Multiplier 2bit

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Multiplier_2bit_tb is
```

```
end Multiplier_2bit_tb;
```

```
architecture Behavioral of Multiplier_2bit_tb is
```

```
-- Component Declaration
```

```
component Multiplier_2bit is
```

```
    Port ( A4 : in STD_LOGIC_VECTOR (3 downto 0);
```

```
          B4 : in STD_LOGIC_VECTOR (3 downto 0);
```

```
          Enable : in STD_LOGIC;
```

```
          Y : out STD_LOGIC_VECTOR (3 downto 0));
```

```
end component;
```

```
-- Signals
```

```
signal A4      : STD_LOGIC_VECTOR (3 downto 0) := "0000";
```

```
signal B4      : STD_LOGIC_VECTOR (3 downto 0) := "0000";
```

```
signal Enable  : STD_LOGIC := '0';
```

```
signal Y       : STD_LOGIC_VECTOR (3 downto 0);
```

```
begin
```

```
-- Instantiate UUT
```

```
uut: Multiplier_2bit
```

```
    PORT MAP (
```

```

        A4 => A4,
        B4 => B4,
        Enable => Enable,
        Y => Y
    );

-- Stimulus Process
stim_proc: process
begin

    -- Enable OFF → output should be 0000
    Enable <= '0';
    A4 <= "0011"; B4 <= "0010"; -- 3 × 2
    wait for 10 ns;

    -- Enable ON
    Enable <= '1';

    -- Test cases (only lower 2 bits used)
    A4 <= "0001"; B4 <= "0001"; wait for 10 ns; -- 1×1 = 1
    A4 <= "0010"; B4 <= "0011"; wait for 10 ns; -- 2×3 = 6
    A4 <= "0011"; B4 <= "0011"; wait for 10 ns; -- 3×3 = 9
    A4 <= "0000"; B4 <= "0011"; wait for 10 ns; -- 0×3 = 0
    A4 <= "0011"; B4 <= "0000"; wait for 10 ns; -- 3×0 = 0

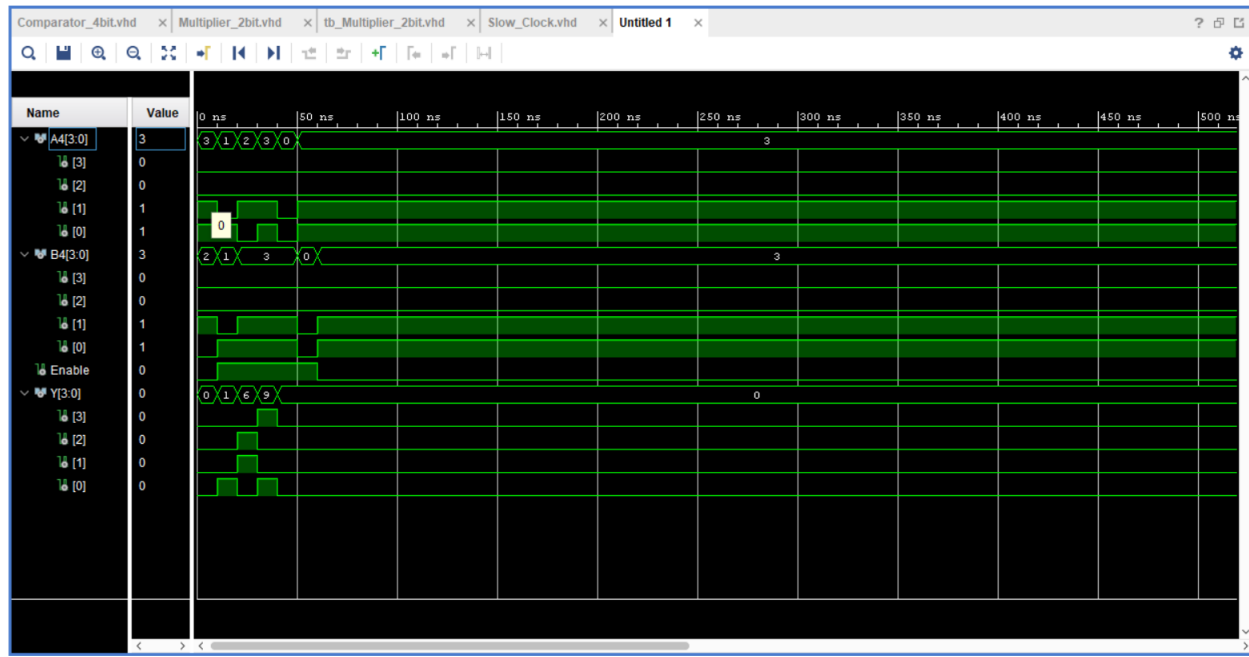
    -- Disable again
    Enable <= '0';
    A4 <= "0011"; B4 <= "0011"; wait for 10 ns;

    -- End simulation
    wait;
end process;

end Behavioral;

```

Timing Diagram for Multiplier 2bit



5. Instruction_decoder_v2

RTL Schematic diagram of Instruction decoder v2

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Instruction_Decoder is
```

```
    Port (
```

```
        I : in STD_LOGIC_VECTOR (12 downto 0); -- 13-bit instruction
```

```
        Reg_Check_Jump : in STD_LOGIC_VECTOR (3 downto 0);
```

```
        Load_Select : out STD_LOGIC;
```

```
        Operation_En : out STD_LOGIC_VECTOR(3 DOWNTO 0);
```

```
        Imm_Value : out STD_LOGIC_VECTOR (3 downto 0);
```

```
        Reg_Enable : out STD_LOGIC_VECTOR (2 downto 0);
```

```
        Reg_Select_1 : out STD_LOGIC_VECTOR (2 downto 0);
```

```
        Reg_Select_2 : out STD_LOGIC_VECTOR (2 downto 0);
```

```
        Add_Sub : out STD_LOGIC;
```

```
        Jump_Flag : out STD_LOGIC;
```

```
        Address : out STD_LOGIC_VECTOR (2 downto 0)
```

```
    );
```

```
end Instruction_Decoder;
```

```
architecture Behavioral of Instruction_Decoder is
```

```
    signal OpCode : STD_LOGIC_VECTOR(2 downto 0);
```

```
    signal RegA : STD_LOGIC_VECTOR(2 downto 0);
```

```
    signal RegB : STD_LOGIC_VECTOR(2 downto 0);
```

```

    signal Imm    : STD_LOGIC_VECTOR(3 downto 0);

begin

    OpCode <= I(12 downto 10);    --used for operations
    RegA <= I(9 downto 7);
    RegB <= I(6 downto 4);
    Imm <= I(3 downto 0);

    Reg_Enable <= "000" when (OpCode(2) = '1') else RegA ;
    Imm_Value <= Imm;
    Address <= I(2 downto 0);

    Load_Select <= '1' when (OpCode = "010") else '0';
    Add_Sub <= '1' when (OpCode = "001") else '0';

    process(OpCode)
    begin
        case OpCode is
            when "111" =>
                Operation_En <= "0100";  -- Multiplier enable
            when "100" =>
                Operation_En <= "0010";  -- Comparator enable
            when "010" =>
                Operation_En <= "1000";
            when others =>
                Operation_En <= "0001";  -- Adder/Sub enable
        (default)
        end case;
    end process;

    process(OpCode, RegB, RegA)
    begin
        if OpCode = "001" then
            Reg_Select_1 <= "000";
            Reg_Select_2 <= RegA;
        else
            Reg_Select_2 <= RegB;
            Reg_Select_1 <= RegA;
        end if;
    end process;

    Jump_Flag <= '1' when (OpCode = "011" and Reg_Check_Jump = "0000")
else '0';

```


RTL Schematic diagram of Instruction decoder v2



```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_Instruction_Decoder_v2 is
end tb_Instruction_Decoder_v2;

architecture Behavioral of tb_Instruction_Decoder_v2 is

    component Instruction_Decoder
    Port (
        I                : in STD_LOGIC_VECTOR (12 downto 0);
        Reg_Check_Jump   : in STD_LOGIC_VECTOR (3 downto 0);
        Load_Select      : out STD_LOGIC;
        Operation_En      : out STD_LOGIC_VECTOR (3 downto 0);
        Imm_Value         : out STD_LOGIC_VECTOR (3 downto 0);
        Reg_Enable        : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_1      : out STD_LOGIC_VECTOR (2 downto 0);
        Reg_Select_2      : out STD_LOGIC_VECTOR (2 downto 0);
        Add_Sub           : out STD_LOGIC;
    );
end component;

-- Component instantiation
    U1 : Instruction_Decoder
        port map (
            I                => I,
            Reg_Check_Jump   => Reg_Check_Jump,
            Load_Select      => Load_Select,
            Operation_En      => Operation_En,
            Imm_Value         => Imm_Value,
            Reg_Enable        => Reg_Enable,
            Reg_Select_1      => Reg_Select_1,
            Reg_Select_2      => Reg_Select_2,
            Add_Sub           => Add_Sub
        );
end Behavioral;

```

```

        Jump_Flag      : out STD_LOGIC;
        Address        : out STD_LOGIC_VECTOR (2 downto 0)
    );
end component;

signal I              : STD_LOGIC_VECTOR(12 downto 0) := (others
=> '0');
signal Reg_Check_Jump : STD_LOGIC_VECTOR(3 downto 0) := (others =>
'0');

signal Load_Select   : STD_LOGIC;
signal Operation_En  : STD_LOGIC_VECTOR(3 downto 0);
signal Imm_Value     : STD_LOGIC_VECTOR(3 downto 0);
signal Reg_Enable    : STD_LOGIC_VECTOR(2 downto 0);
signal Reg_Select_1  : STD_LOGIC_VECTOR(2 downto 0);
signal Reg_Select_2  : STD_LOGIC_VECTOR(2 downto 0);
signal Add_Sub       : STD_LOGIC;
signal Jump_Flag     : STD_LOGIC;
signal Address       : STD_LOGIC_VECTOR(2 downto 0);

begin

    uut: Instruction_Decoder PORT MAP (
        I => I,
        Reg_Check_Jump => Reg_Check_Jump,
        Load_Select => Load_Select,
        Operation_En => Operation_En,
        Imm_Value => Imm_Value,
        Reg_Enable => Reg_Enable,
        Reg_Select_1 => Reg_Select_1,
        Reg_Select_2 => Reg_Select_2,
        Add_Sub => Add_Sub,
        Jump_Flag => Jump_Flag,
        Address => Address
    );

    stim_proc: process
    begin
        wait for 10 ns;

        I <= "00000101000000";
        Reg_Check_Jump <= "0000";
        wait for 20 ns;

        I <= "00101100000000";

```

```

wait for 20 ns;

I <= "0101100001010";
wait for 20 ns;

I <= "0111110000101";
Reg_Check_Jump <= "1111";
wait for 20 ns;

Reg_Check_Jump <= "0000";
wait for 20 ns;

I <= "1001111100000";
wait for 20 ns;

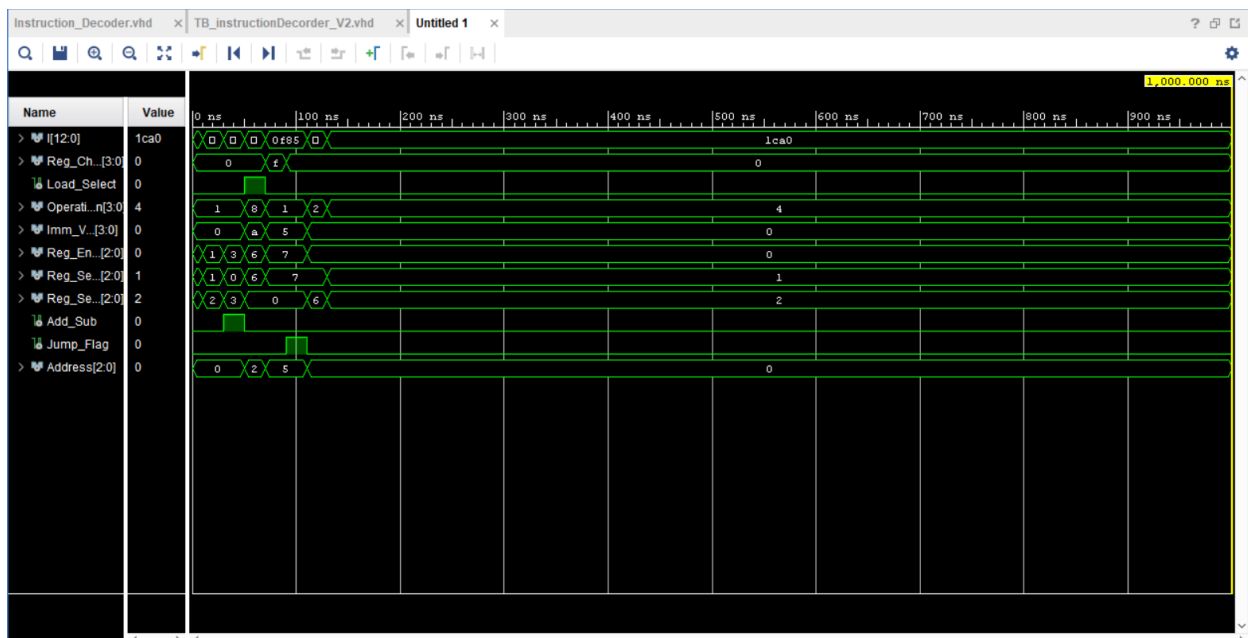
I <= "1110010100000";
wait for 20 ns;

wait;
end process;

```

end Behavioral;

Timing Diagram for Instruction decoder v2



6. ROM_v2

Design source VHDL code of ROM v2

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity Program_ROM is
    Port ( address : in STD_LOGIC_VECTOR (2 downto 0);
          instruction_code : out STD_LOGIC_VECTOR (12 downto 0));
end Program_ROM;

architecture Behavioral of Program_ROM is

    type rom_type is array (0 to 7) of std_logic_vector(12 downto 0);

    signal twelveSegmant_ROM : rom_type := ( -- Program is hardcoded
    here.

    -- TASK 03 : Extra Functionality => 4bit Comparator and 2bit
    Multiplier

        "01000100000010", -- Movi R1, 2
        "01001000000011", -- Movi R2, 3
        "01001100000011", -- Movi R3, 3
        "1000100010000", -- Compare R2, R1
        "1110110100000", -- Multiply R3, R2
        "1000010100000", -- Compare R1, R2
        "1110110010000", -- Multiply R3, R1
        "1000100110000" -- Compare R2, R3

    );

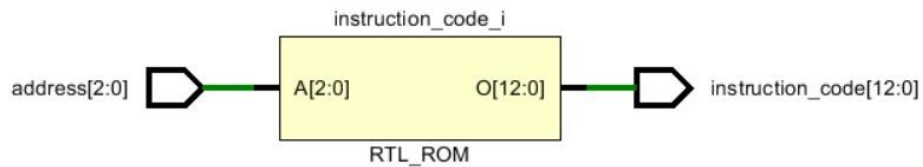
begin

    instruction_code <= twelveSegmant_ROM(to_integer(unsigned(address)));

end Behavioral;

```

RTL Schematic diagram of Instruction ROM v2



Behavioral simulation Code for Instruction ROM v2

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Program_ROM_tb is
end Program_ROM_tb;

architecture Behavioral of Program_ROM_tb is

    -- Component Declaration
    component Program_ROM
        Port (
            address : in STD_LOGIC_VECTOR (2 downto 0);
            instruction_code : out STD_LOGIC_VECTOR (12 downto 0)
        );
    end component;

    -- Signals
    signal address : STD_LOGIC_VECTOR (2 downto 0) := (others => '0');
    signal instruction_code : STD_LOGIC_VECTOR (12 downto 0);

begin

    -- Instantiate the Unit Under Test (UUT)
    uut: Program_ROM
        port map (
            address => address,
            instruction_code => instruction_code
        );

```

```

-- Test Process
stim_proc: process
begin

    -- Address 0
    address <= "000";
    wait for 10 ns;

    -- Address 1
    address <= "001";
    wait for 10 ns;

    -- Address 2
    address <= "010";
    wait for 10 ns;

    -- Address 3
    address <= "011";
    wait for 10 ns;

    -- Address 4
    address <= "100";
    wait for 10 ns;

    -- Address 5
    address <= "101";
    wait for 10 ns;

    -- Address 6
    address <= "110";
    wait for 10 ns;

    -- Address 7
    address <= "111";
    wait for 10 ns;

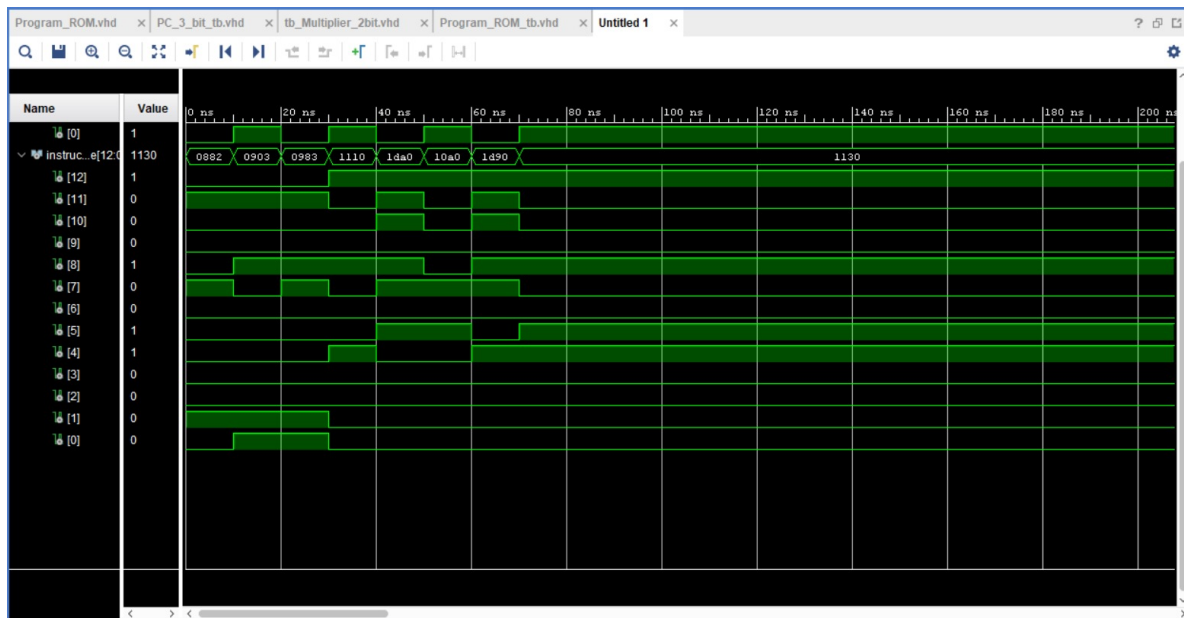
    wait;

end process;

end Behavioral;

```

Timing Diagram for Instruction ROM v2



7. Program Counter_v2

Design source VHDL code of Programme Counter v2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity PC_3_bit is
    Port ( D : in STD_LOGIC_VECTOR (2 downto 0);
          Res : in STD_LOGIC;
          Clk : in STD_LOGIC;
          Q : out STD_LOGIC_VECTOR (2 downto 0));
end PC_3_bit;

architecture Behavioral of PC_3_bit is
    component D_FF is
        Port ( D : in STD_LOGIC;
              Res : in STD_LOGIC;
              Clk : in STD_LOGIC;
              Q : out STD_LOGIC);
    end component;

    signal Res_Sig : STD_LOGIC := '1';

begin

    DFF_0 : D_FF
        PORT MAP (D => D(0), Q => Q(0), Clk => Clk, Res => Res_Sig);
```

```

DFF_1 : D_FF
  PORT MAP (D => D(1), Q => Q(1), Clk => Clk, Res => Res_Sig);

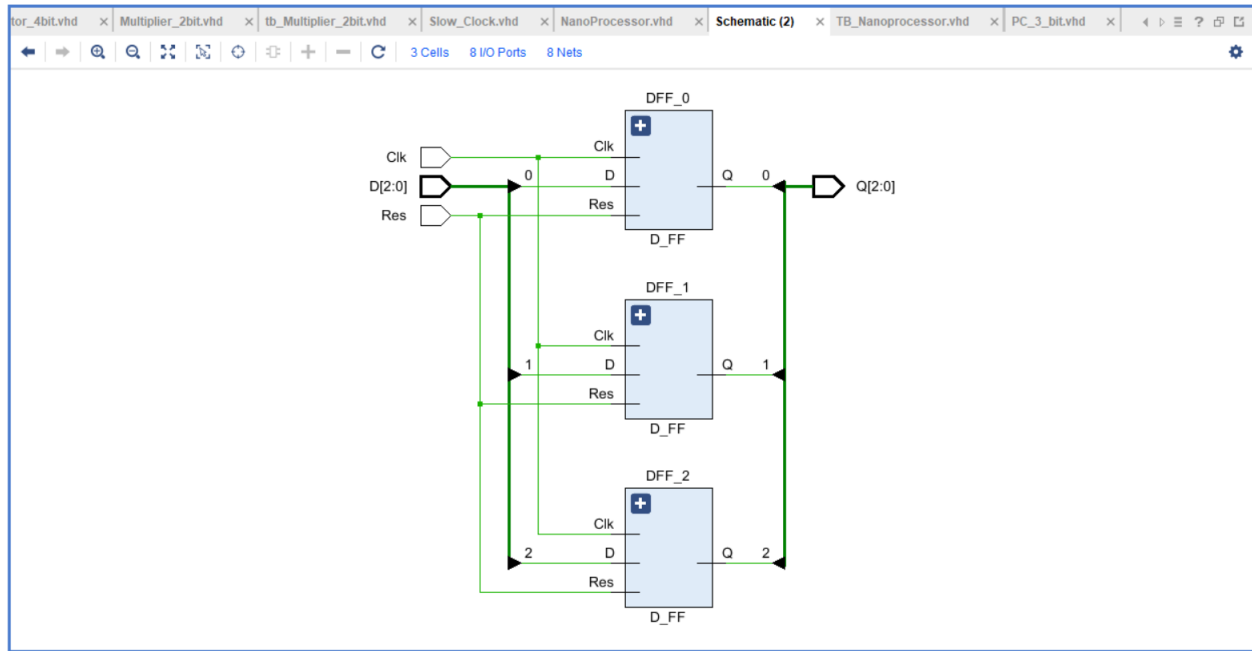
DFF_2 : D_FF
  PORT MAP (D => D(2), Q => Q(2), Clk => Clk, Res => Res_Sig);

Res_Sig <= Res;

end Behavioral;

```

RTL Schematic diagram of Instruction Programme Counter v2



Behavioral simulation Code for Programme Counter v2

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PC_3_bit_tb is
end PC_3_bit_tb;

architecture Behavioral of PC_3_bit_tb is

component PC_3_bit
  Port ( D : in STD_LOGIC_VECTOR (2 downto 0);
        Res : in STD_LOGIC;
        Clk : in STD_LOGIC;
        Q : out STD_LOGIC_VECTOR (2 downto 0));
end component;

```



```
signal D    : STD_LOGIC_VECTOR (2 downto 0) := "000";
signal Res  : STD_LOGIC := '0';
signal Clk  : STD_LOGIC := '0';
signal Q    : STD_LOGIC_VECTOR (2 downto 0);
```

```
begin
```

```
UUT : PC_3_bit
  PORT MAP (
    D    => D,
    Res => Res,
    Clk  => Clk,
    Q    => Q
  );
```

```
clk_process : process
begin
  while true loop
    Clk <= '0';
    wait for 5 ns;
    Clk <= '1';
    wait for 5 ns;
  end loop;
end process;
```

```
stim_proc : process
begin
  Res <= '1';
  wait for 10 ns;
  Res <= '0';

  D <= "001"; wait for 10 ns;
  D <= "010"; wait for 10 ns;
  D <= "011"; wait for 10 ns;
  D <= "100"; wait for 10 ns;
  D <= "101"; wait for 10 ns;

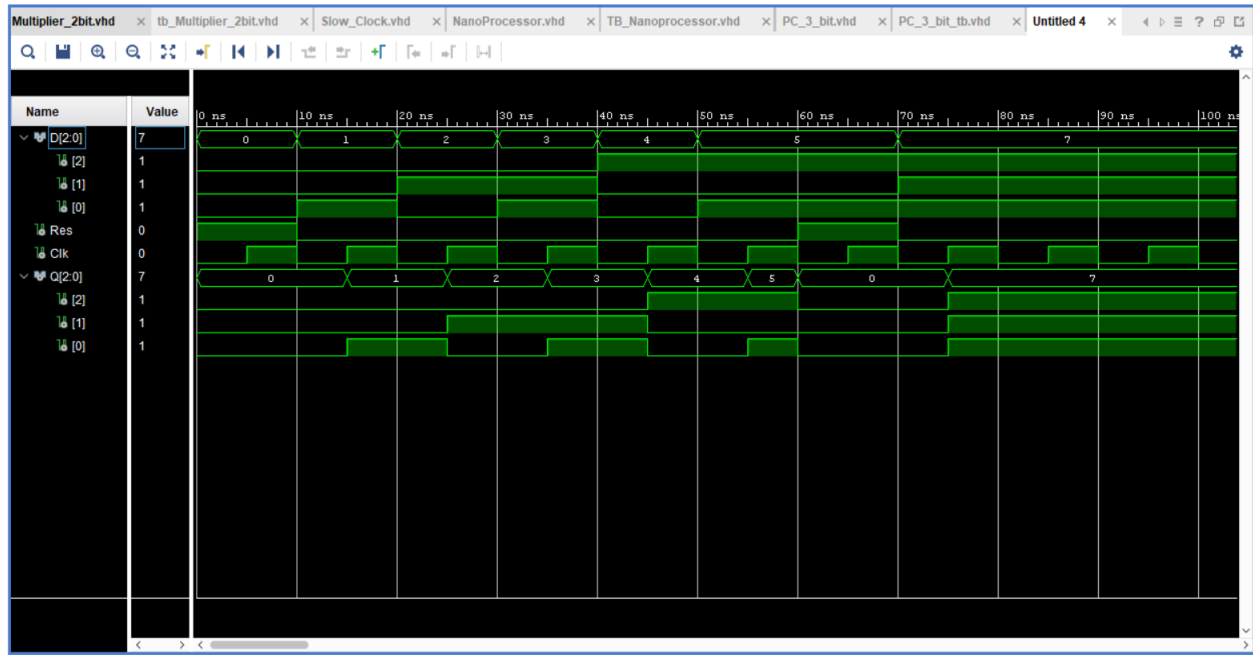
  Res <= '1'; wait for 10 ns;
  Res <= '0';

  D <= "111"; wait for 10 ns;

  wait;
end process;
```

```
end Behavioral;
```

Timing Diagram for Programme Counter v2



8. Add_Sub4bit

Design source VHDL code of Add Sub4bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ADD_SUB_4bit is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);      -- A and B 4 bit
          Enable : in STD_LOGIC;
          Operation : in STD_LOGIC;                  -- ADD - '0' and
          SUB - '1'
          C_out : out STD_LOGIC;
          S : out STD_LOGIC_VECTOR (3 downto 0);
          Overflow : out STD_LOGIC;
          Zero : out STD_LOGIC);                      -- '1', if the
total result is zero
end ADD_SUB_4bit;

architecture Behavioral of ADD_SUB_4bit is
```

```

component RCA_4
    PORT(A : in STD_LOGIC_VECTOR(3 downto 0);
          B : in STD_LOGIC_VECTOR(3 downto 0);
          C_in : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR(3 downto 0);
          C_out : out STD_LOGIC);
end component;

SIGNAL Y,X : STD_LOGIC_VECTOR(3 downto 0);
begin
X(0) <= B(0) XOR Operation;
X(1) <= B(1) XOR Operation;
X(2) <= B(2) XOR Operation;
X(3) <= B(3) XOR Operation;

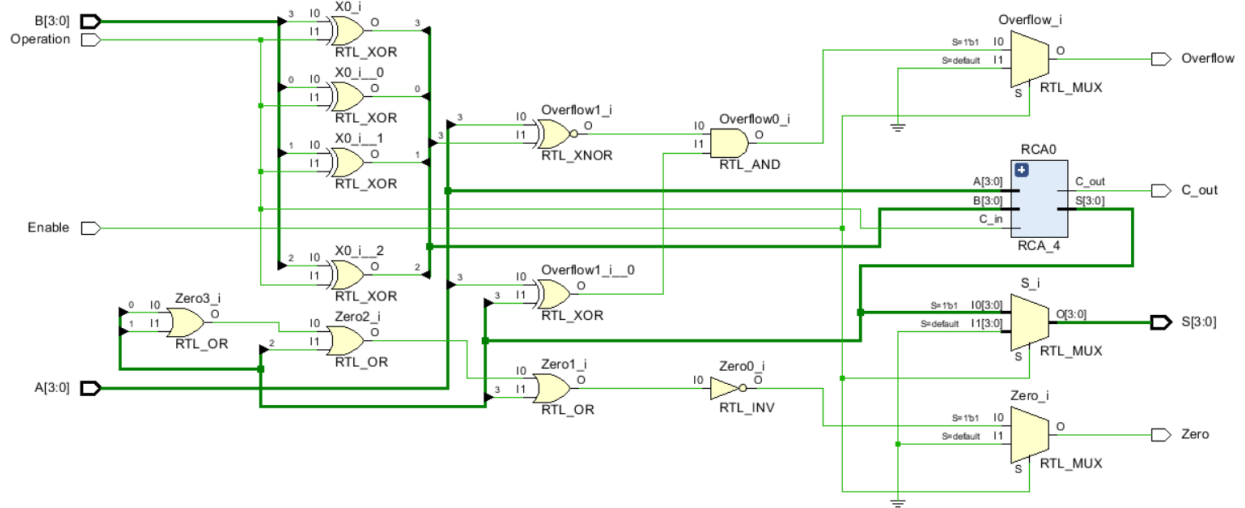
RCA0 : RCA_4
    port map(
        A => A,
        B => X,
        C_in => Operation,
        S => Y,
        C_out => C_out);

S <= Y when Enable = '1' else (others => '0');
Overflow <= ( (A(3) XNOR (Operation XOR B(3))) AND (A(3) XOR Y(3)) )
when Enable = '1' else '0';
Zero <= ( NOT(Y(0) OR Y(1) OR Y(2) OR Y(3)) ) when Enable = '1' else
'0';

end Behavioral;

```

RTL Schematic diagram of Add Sub4bit



Behavioral simulation Code for Add Sub4bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_ADD_SUB is
end TB_ADD_SUB;

architecture Behavioral of TB_ADD_SUB is

    component ADD_SUB_4bit
        Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
              B : in STD_LOGIC_VECTOR (3 downto 0);
              Operation : in STD_LOGIC;
              C_out : out STD_LOGIC;
              S : out STD_LOGIC_VECTOR (3 downto 0);
              Overflow : out STD_LOGIC;
              Zero : out STD_LOGIC);
    end component;

    -- Inputs
    signal A : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal B : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal Operation : STD_LOGIC := '0';

    -- Outputs
    signal C_out : STD_LOGIC;
    signal S : STD_LOGIC_VECTOR(3 downto 0);
```

```

    signal Overflow : STD_LOGIC;
    signal Zero : STD_LOGIC;

begin

    UUT: ADD_SUB_4bit port map (
        A => A,
        B => B,
        Operation => Operation,
        C_out => C_out,
        S => S,
        Overflow => Overflow,
        Zero => Zero
    );

    stim_proc: process
    begin
        -- ---- ADDITION TESTS (Operation = '0') ----
        Operation <= '0';

        -- 2 + 3 = 5
        A <= "0010"; B <= "0011";
        wait for 20 ns;

        -- 7 + 1 = 8 (Signed Overflow Expected: Positive + Positive =
Negative)
        A <= "0111"; B <= "0001";
        wait for 20 ns;

        -- ---- SUBTRACTION TESTS (Operation = '1') ----
        Operation <= '1';

        -- 5 - 2 = 3
        A <= "0101"; B <= "0010";
        wait for 20 ns;

        -- 4 - 4 = 0 (Zero Flag Expected)
        A <= "0100"; B <= "0100";
        wait for 20 ns;

        -- -4 - 5 = -9 (Signed Overflow Expected)
        A <= "1100"; B <= "0101";
        wait for 20 ns;

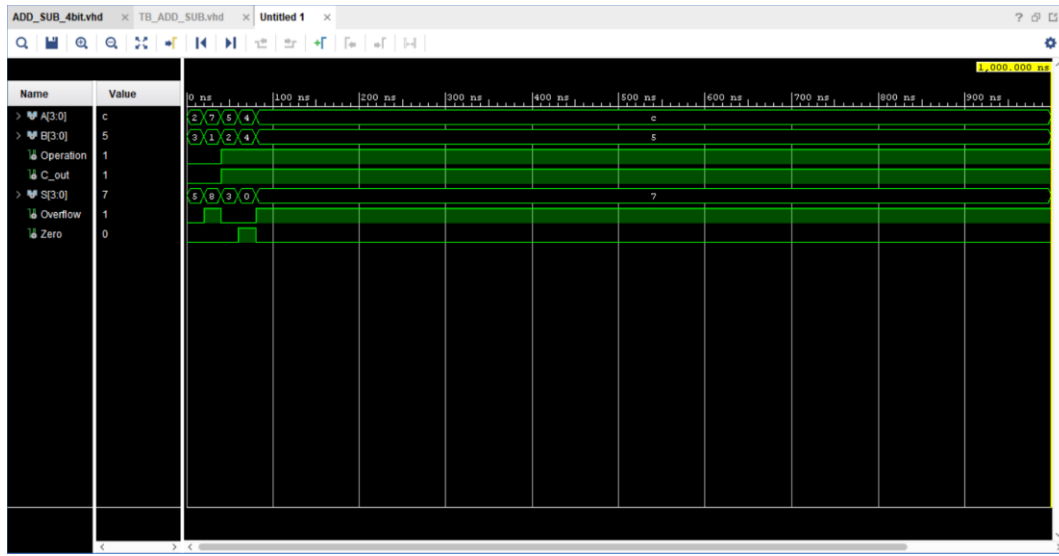
        wait;
    end process;
end;

```

```
end process;
```

```
end Behavioral;
```

Timing Diagram for Add Sub4bit



Constraints File for Version 2

```
## Clock
```

```
set_property PACKAGE_PIN W5 [get_ports clk]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

```
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}  
[get_ports clk]
```

```
set_property PACKAGE_PIN U18 [get_ports reset]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports reset]
```

```
## reg[3:0] at' LEDs
```

```
set_property PACKAGE_PIN U16 [get_ports {reg[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {reg[0]}]
```

```
set_property PACKAGE_PIN E19 [get_ports {reg[1]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {reg[1]}]
```

```
set_property PACKAGE_PIN U19 [get_ports {reg[2]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {reg[2]}]
```

```
set_property PACKAGE_PIN V19 [get_ports {reg[3]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {reg[3]}]
```

```

set_property PACKAGE_PIN V3 [get_ports {var0[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var0[0]}]
set_property PACKAGE_PIN W3 [get_ports {var0[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var0[1]}]
set_property PACKAGE_PIN U3 [get_ports {var0[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var0[2]}]
set_property PACKAGE_PIN P3 [get_ports {var0[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var0[3]}]

set_property PACKAGE_PIN U15 [get_ports {var1[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var1[0]}]
set_property PACKAGE_PIN U14 [get_ports {var1[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var1[1]}]
set_property PACKAGE_PIN V14 [get_ports {var1[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var1[2]}]
set_property PACKAGE_PIN V13 [get_ports {var1[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {var1[3]}]

## overflow, zero, carry â€˜ status indicators
set_property PACKAGE_PIN N3 [get_ports {overflow}]
set_property IOSTANDARD LVCMOS33 [get_ports {overflow}]
set_property PACKAGE_PIN P1 [get_ports {zero}]
set_property IOSTANDARD LVCMOS33 [get_ports {zero}]
set_property PACKAGE_PIN L1 [get_ports {carry}]
set_property IOSTANDARD LVCMOS33 [get_ports {carry}]

## 7-Segment Display segments: display[6:0]
set_property PACKAGE_PIN W7 [get_ports {display[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[0]}]
set_property PACKAGE_PIN W6 [get_ports {display[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[1]}]
set_property PACKAGE_PIN U8 [get_ports {display[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[2]}]
set_property PACKAGE_PIN V8 [get_ports {display[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[3]}]
set_property PACKAGE_PIN U5 [get_ports {display[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[4]}]
set_property PACKAGE_PIN V5 [get_ports {display[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[5]}]
set_property PACKAGE_PIN U7 [get_ports {display[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {display[6]}]

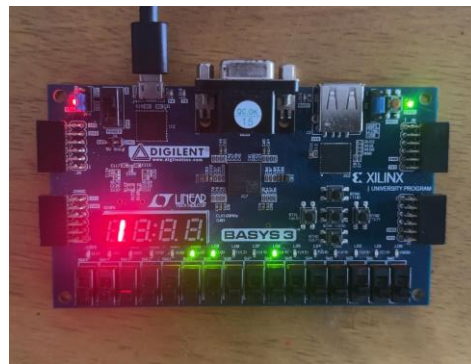
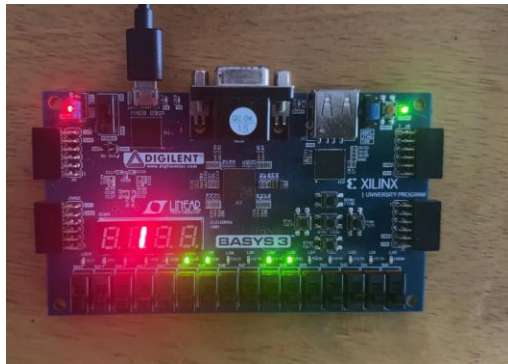
## Anodes: anode[3:0]
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]

```

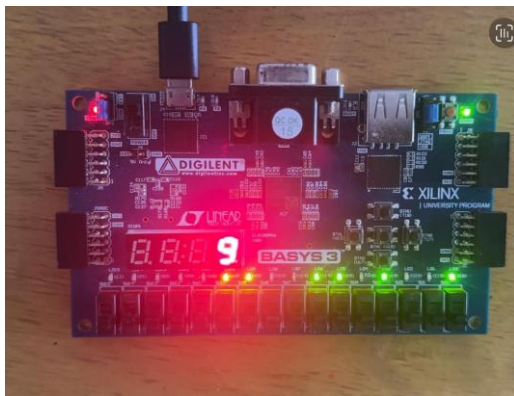
```
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]
```

Tasks Completed in Version 2

Comparation of Numbers



Multiplier



Future Suggestions:

- In our extended instruction set, there are some instructions we didn't use for the design and those can be used to extend the design with extra functionalities.

- Additionally, based on the type of instruction, enable inputs and construct so that the input directs to the necessary operation unit only
- Process the seven-bit output resulting from the multiplication of two four-bit numbers and perform a complete multiplication operation.
- Add RAM in addition to ROM to allow dynamic read/write operations.
- Create a simple Python or C-based assembler that converts symbolic assembly to binary code.

Conclusion:

For this lab exercise, we successfully designed and constructed two versions of a simple 4-bit nano processor, each version enhanced to enable a wider instruction set and broader architectural capability. Version 1 was initially developed with a 3-bit program counter, a basic Arithmetic Logic Unit that can only do addition and subtraction, and four basic core instructions MOVI, ADD, NEG, and JZR, enabling us to test basic computation and control functions in a constrained environment. Building on this, the Version 2 nano processor included a 4-bit program counter and provided instruction set functions for AND, OR, XOR, NOT bitwise logical operations, 4 bit comparison, limited multiplication only 2bit, and right shifts by 2 or 1 positions. These enhancements significantly improved the functional expressiveness of our processor while maintaining a compact design suitable for FPGA deployment. Through this project, we had hands-on experience with hardware description languages (VHDL), instruction decoding, modular digital design, and system-level integration. The team effort of designing, simulating, and testing the parts collectively provided us with a deeper understanding of digital logic pipelines and imposed necessary engineering disciplines like planning, debugging, and documentation. Lastly, the lab emphasized how simple, well-structured hardware components can be composed into a working processor. The iterative improvement between Version 1 and Version 2 also demonstrated the real-world process of architectural scaling and feature expansion in CPU design. This exercise served as a valuable stepping 140 stone toward understanding modern microprocessor architecture and embedded systems.

Contribution:

This project is a reflection of the consistent hard work, determination, and effective teamwork demonstrated by all members throughout its development.

- ❖ We began by engaging in detailed group discussions to thoroughly understand the problem and to plan how each component of the Nano-processor would function within the system.

❖ Task responsibilities were distributed based on each member's strengths and interests, allowing us to focus on designing and building individual components efficiently.

❖ Once the components were developed, we collaborated closely to integrate them seamlessly into the main CPU file, ensuring system compatibility and performance.

❖ Through continuous refinement and collective problem-solving, we successfully developed an improved version of the Nano-processor with enhanced functionality.

❖ To conclude our efforts, we compiled a comprehensive lab report, clearly documenting each phase of the project, our methodologies, challenges faced, and the final outcomes.

Members	Individual Contribution	Number of hours spent
M.Dinojan	• 2 way 3 bit multiplexer • 2 way 4 bit multiplexer • 8 way 4 bit multiplexer • Comparators for version 2 • Test bench files for the above components • Code debugging and testing on board	25 hours
De Zoysa P.C.A	• Program ROM V1 & V2 • Design and port mapping of Nano processor V1 & V2. • Multiplier V2 • Test bench files of Nano processor V1 & V2. • Test bench files of ROM v1 & v2 • Lab report	28 hours
Dhananjaya P.D.L.K	• Decoder 3 to 8 V1	25 hours

	• 4 bit register V1 • Register bank V1 • Instruction Decoder V1 & V2 • Test bench files of Decoder 3 to 8 V1 • Test bench files of Instruction Decoder V1 & V2 • prepared a presentation slide	
W.W.N.D.Wickramarathna	• Add/Sub unit • FA/HA • RCA • 3 bit Adder • D flip-flop • Test bench files for Add/Sub unit • Test bench files for 3 bit adder • Programme counter	20 hours